UNIVERSIDAD COOPERATIVA DE COLOMBIA

FARMTOTABLE

Marketplace de productos agrícolas directos

Momento 1

Diseño arquitectónico y estrategia DevOps

Arquitectura y Modelamiento de Software

Karol Melissa Figueroa

Rafael Sibaja

Andrés Suárez

6 de septiembre de 2026

Contenido

1. Estilo arquitectónico 3

2. Arquitectura general 4

3. Responsabilidades y contratos 4

3.9. Identidad y Acceso 5

4. Modelo de datos por microservicio 6

4.1. Productores 7

4.2. Catálogo y Cosechas 8

4.3. Pedidos 10

4.4. Inventario Perecedero 13

4.5. Logística y Rutas 16

4.6. Pagos 20

4.7. Notificaciones 24

4.8. Analítica 26

4.9. Identidad y Acceso 29

4.10. Persistencia de eventos 32

5. Comunicación y operación del negocio 33

5.4. Consolidación por zona y ventanas 35

5.5. App de transportistas en React Native 36

5.6. Indicadores con fórmula y fuente 36

6. Diseño del API Gateway y acceso 37

7. Estrategia DevOps 38

8. Pipeline CI/CD con GitHub Actions 39

9. Estrategia de despliegue 43

10. Observabilidad y monitoreo 45

11. Seguridad arquitectónica 45

1. Estilo arquitectónico

FarmToTable conecta productores con restaurantes, tiendas y consumidores. Se divide en nueve microservicios con bases PostgreSQL independientes: ocho servicios de negocio y uno de Identidad y Acceso. La separación permite escalar las consultas de Catálogo, las reservas de Inventario y el trabajo de Logística según su carga, sin convertirlos en una sola aplicación.

REST se utiliza para respuestas inmediatas y RabbitMQ para cambios que otros servicios procesan después. El Gateway es la entrada pública; cada servicio es dueño de sus datos. La consistencia entre servicios es eventual, con eventos idempotentes y compensaciones. Una operación local usa una transacción de PostgreSQL; no existe una transacción única que abarque todas las bases.

1.1. Ventajas, costos y principios

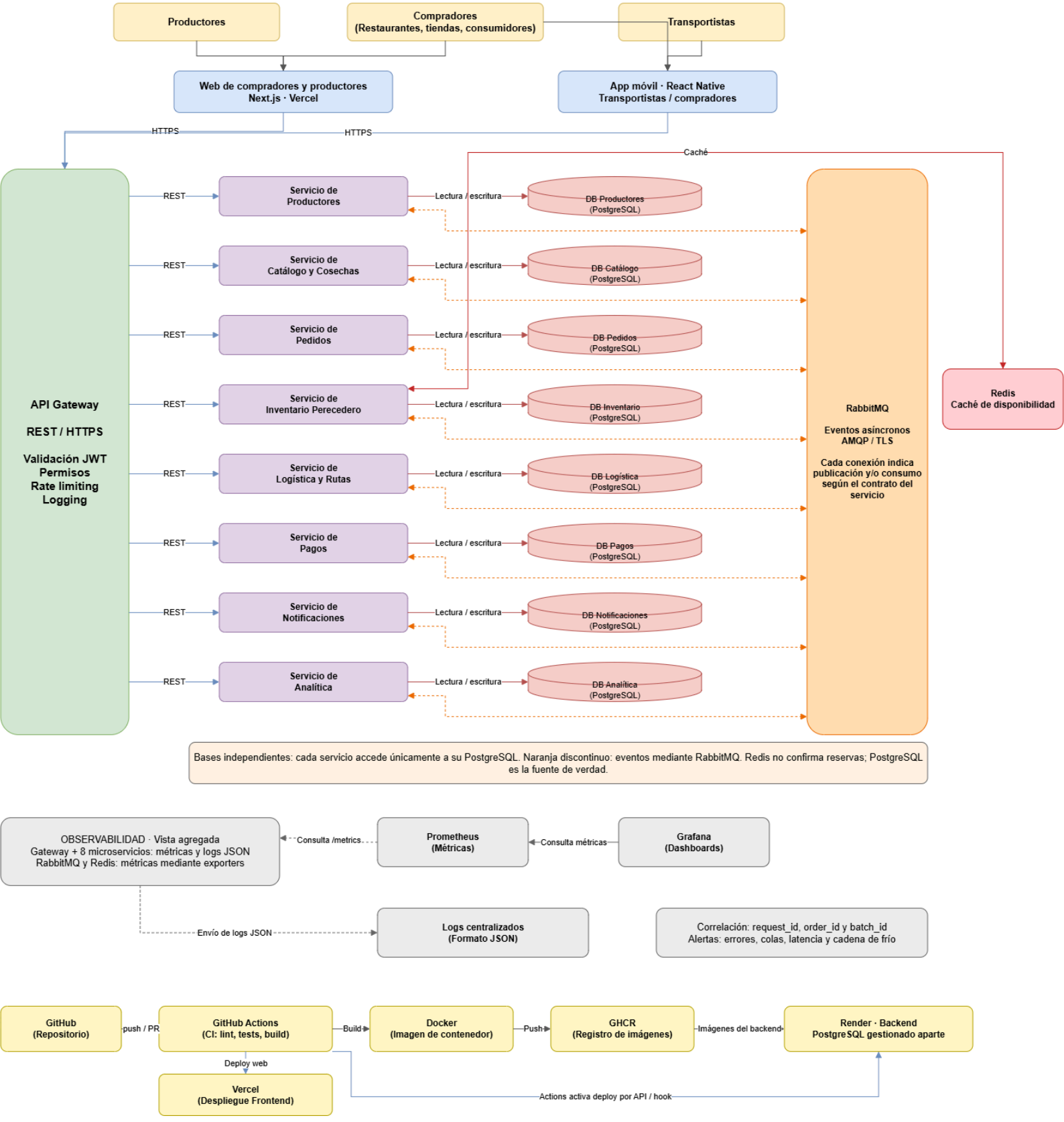
Decisión	Justificación y límite
Separación por responsabilidad	Un fallo de Analítica no debe impedir comprar. La comunicación por red añade latencia y exige timeouts y monitoreo.
Base propia por servicio	Evita escrituras ajenas y acoplamiento de tablas. Los identificadores externos se validan por API o eventos; no se hacen joins entre bases.
SOLID y contratos	Responsabilidad única por servicio; interfaces pequeñas y versionadas; pasarela de pago intercambiable; dependencias a través de contratos.
Saga y eventos	Permiten confirmar reserva y autorización, o compensar lo que sí ocurrió. Repetir un mensaje no debe repetir el efecto.

1.2. Redis es una caché

Redis conserva una copia de disponibilidad para acelerar el catálogo. Se propone TTL de 30 segundos e invalidación con inventory.stock_changed. Una lectura puede estar atrasada: mostrar “disponible” no garantiza una compra. Si Redis falla, se consulta Inventario; no se pierde el saldo real.

La reserva definitiva ocurre en inventory_db: Inventario bloquea las filas de lotes, verifica vencimiento y saldo, selecciona FEFO y actualiza reserva y cantidades en la misma transacción. Solo después del commit publica inventory.reserved mediante outbox. No se confía en un contador ni en un bloqueo de Redis para impedir sobreventa.

FarmToTable - Diagrama de Arquitectura General



3.3. Pedidos

Carrito, condiciones aceptadas, estado físico/financiero y cierre. Endpoints: /cart; /orders; /orders/{id}/ratings. Publica order.created/confirmed/cancelled, order.fulfillment_closed y rating.created. Consume reserva, autorización, captura y estados de entrega.

3.4. Inventario

Existencia real, FEFO, reservas, movimientos y mermas. Endpoints: /inventory/products/{id}; /inventory/reservations; /inventory/waste. Publica inventory.reserved/insufficient/released, inventory.stock_changed, inventory.movement_recorded, inventory.waste_recorded y stock.low. Consume harvest.registered, order.created/cancelled y despachos.

3.5. Logística

Zonas, transportistas, vehículos, paradas, entregas y temperatura. Endpoints: /logistics/zones; /routes; /deliveries; /drivers; /vehicles. Publica delivery.scheduled/in_transit/completed/partial/failed/cancelled y cold_chain.alert. Consume inventory.reserved, order.confirmed/cancelled.

3.6. Pagos

Autorizar, capturar, anular autorización, reembolsar y liquidar. Endpoints: /payments; /payments/order/{id}; /refunds; /payouts. Publica payment.authorized/completed/failed/voided/refunded y payout.scheduled/completed. Consume order.created/cancelled y order.fulfillment_closed.

3.7. Notificaciones

Avisos y preferencias, sin bloquear pedidos. Endpoints: /notifications/user/{id}; /notification-preferences. Consume eventos relevantes de pedido, pago, entrega, producto e identidad. Publica notification.sent/failed.

3.8. Analítica

Proyecciones de ventas, mermas, rotación, ventanas y satisfacción. Endpoints: /analytics/sales; /waste; /inventory-rotation; /deliveries; /satisfaction. Consume hechos financieros, lotes, movimientos, saldos, estados de entrega y rating.created. Publica report.generated.

3.9. Identidad y Acceso

Responsabilidad: registrar y administrar cuentas, verificar credenciales, emitir y renovar tokens, conservar sesiones, revocar accesos y administrar roles. Se incorpora como noveno microservicio, junto a los ocho servicios de negocio existentes.

Tecnología y datos: Node.js con NestJS y PostgreSQL independiente (identity_db). Solo este servicio puede leer o escribir credenciales, roles y sesiones. Productores y Logística mantienen sus perfiles operativos; Pedidos conserva compras. El Gateway no tiene base de cuentas ni la clave privada de firma.

Administración de cuentas

El registro público crea exclusivamente el rol buyer. Un administrador de cuentas (admin) habilita el rol producer después de revisar el perfil, y asigna driver al transportista registrado por Logística. logistics_admin administra vehículos, rutas y perfiles de transportistas, pero no concede roles de acceso. Una cuenta puede tener varios roles. El primer admin se crea mediante una tarea de despliegue de acceso restringido; no hay un endpoint público de promoción a administrador.

Separación de responsabilidades

Componente	Función concreta
Identidad y Acceso	Verifica contraseña, firma JWT, rota refresh tokens, revoca sesiones y administra cuentas y roles.
API Gateway	Enruta /auth y /users, valida firma y claims, consulta sesión vigente y aplica límites. No genera tokens ni guarda contraseñas.
Cada microservicio	Verifica el contexto autenticado y autoriza el recurso: propietario del pedido, productor de la finca o conductor de la ruta.
Productores y Logística	Relacionan sus perfiles locales con USUARIOS.id mediante user_id. No crean credenciales de forma paralela.

Fuente única de identificadores

Identificador	Origen y uso
user_id	identity_db.USUARIOS.id, UUID estable. Se transporta en el claim sub. NOTIFICACIONES.user_id y las referencias de cuentas apuntan a él.
buyer_id	El mismo USUARIOS.id con rol buyer. Pedidos lo toma de sub; no confía en un buyer_id libre enviado por el cliente. Pagos lo recibe del pedido validado.
producer_id	PRODUCTORES.id, no USUARIOS.id. PRODUCTORES.user_id (REF, NOT NULL, UNIQUE) identifica la cuenta propietaria.
driver_id	TRANSPORTISTAS.id en logistics_db. TRANSPORTISTAS.user_id (REF, NOT NULL, UNIQUE) identifica la cuenta con rol driver. RUTAS.driver_id es FK local.

El correo de PRODUCTORES es contacto comercial; USUARIOS.email es el identificador de acceso. Cambiar el primero no cambia las credenciales. Desactivar una cuenta conserva sus referencias históricas y bloquea nuevas acciones autenticadas.

Comunicación y eventos

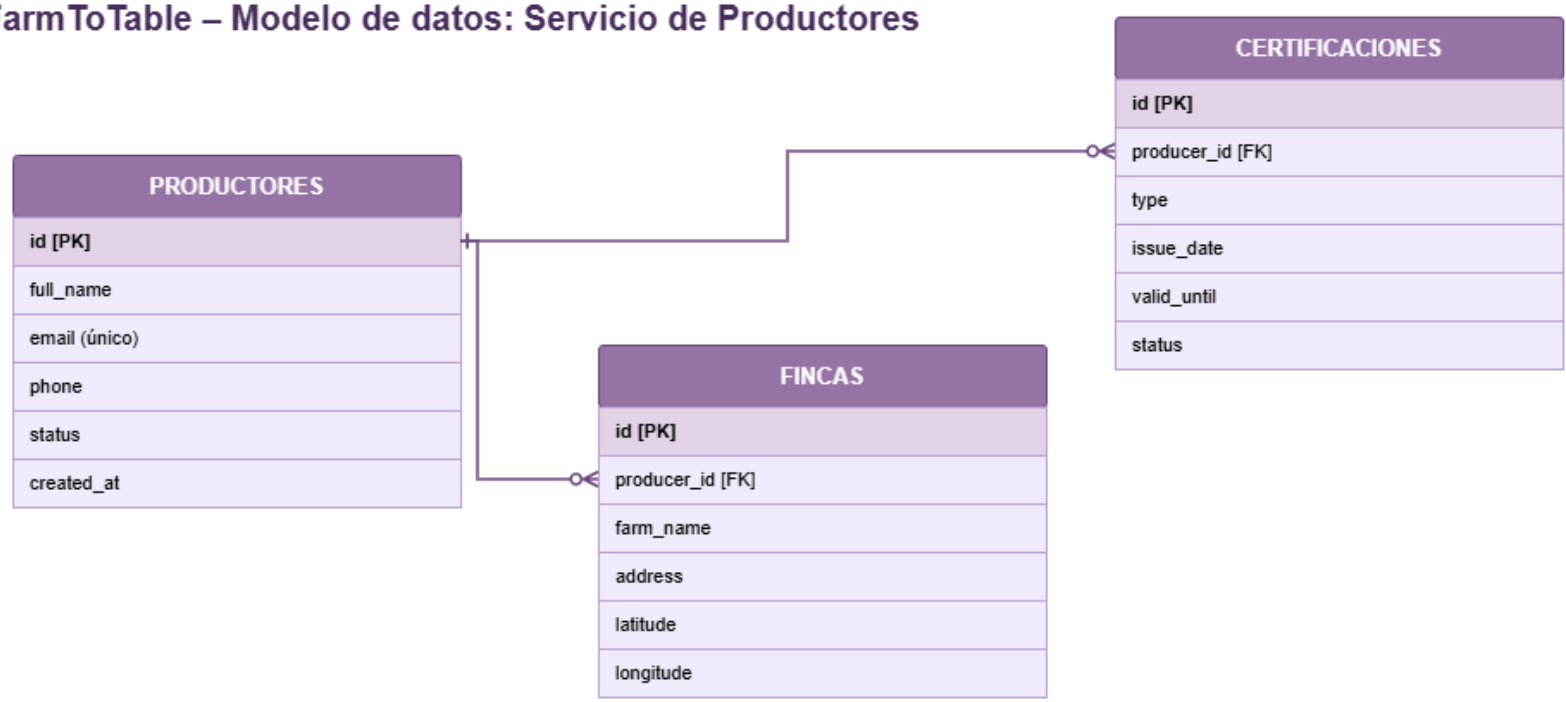
REST para registro, login, gestión y validación interna de sesiones. Publica identity.user_created, identity.user_disabled, identity.roles_changed y identity.session_revoked mediante la outbox definida en 4.10. Cada evento lleva event_id, entity_id, entity_version, occurred_at y correlation_id; además user_id y, cuando aplica, session_id o roles. Nunca incluye contraseña, hash, JWT ni refresh token. Productores y Logística consumen las bajas; Notificaciones consume altas y cambios para avisos informativos. Identidad no consume eventos de negocio para conceder permisos automáticamente. Los correos de verificación y recuperación se envían desde un adaptador de correo de Identidad por HTTPS; los secretos de recuperación no viajan por RabbitMQ.

4. Modelo de datos por microservicio

PK significa clave primaria; FK es una relación dentro de la misma base; REF identifica datos de otro servicio. “No” significa NOT NULL. Todas las PK son únicas e indexadas. Las FK usan ON DELETE RESTRICT, salvo excepción explícita, y no permiten uniones entre bases. Los índices adicionales se indican por tabla. Los CHECK afectan una fila; sumas, límites de varias filas y estados coordinados se validan en transacciones con bloqueo.

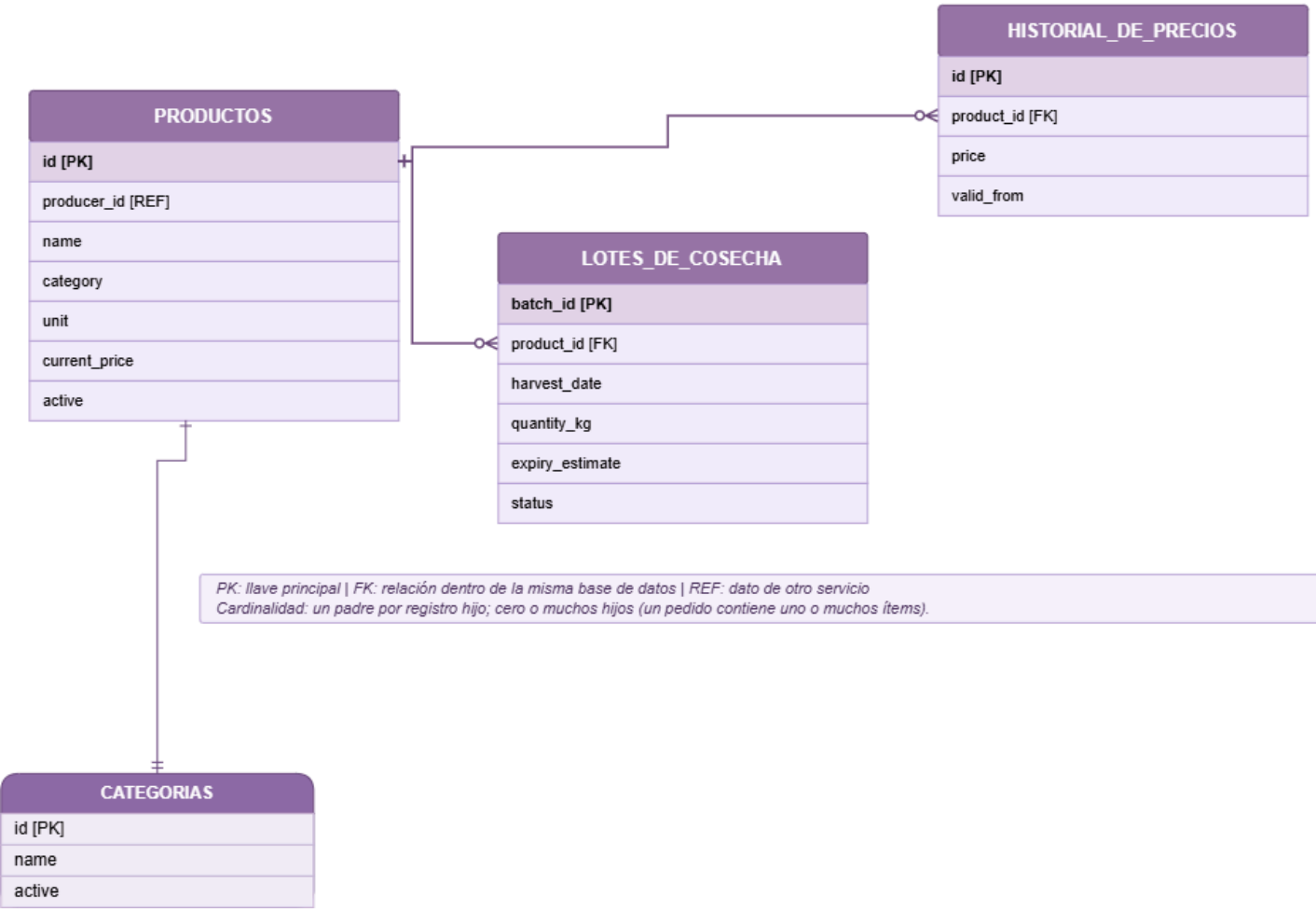
Se usan kg y COP en todo este alcance, numeric para cantidades/importes y timestamptz para instantes UTC. Los índices y nulabilidad son decisiones del modelo, no migraciones ya ejecutadas. Las copias deliberadas conservan historia o aceleran lectura; el propietario de cada dato sigue siendo único.

FarmToTable – Modelo de datos: Servicio de Productores



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

FarmToTable – Modelo de datos: Servicio de Catálogo y Cosechas



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

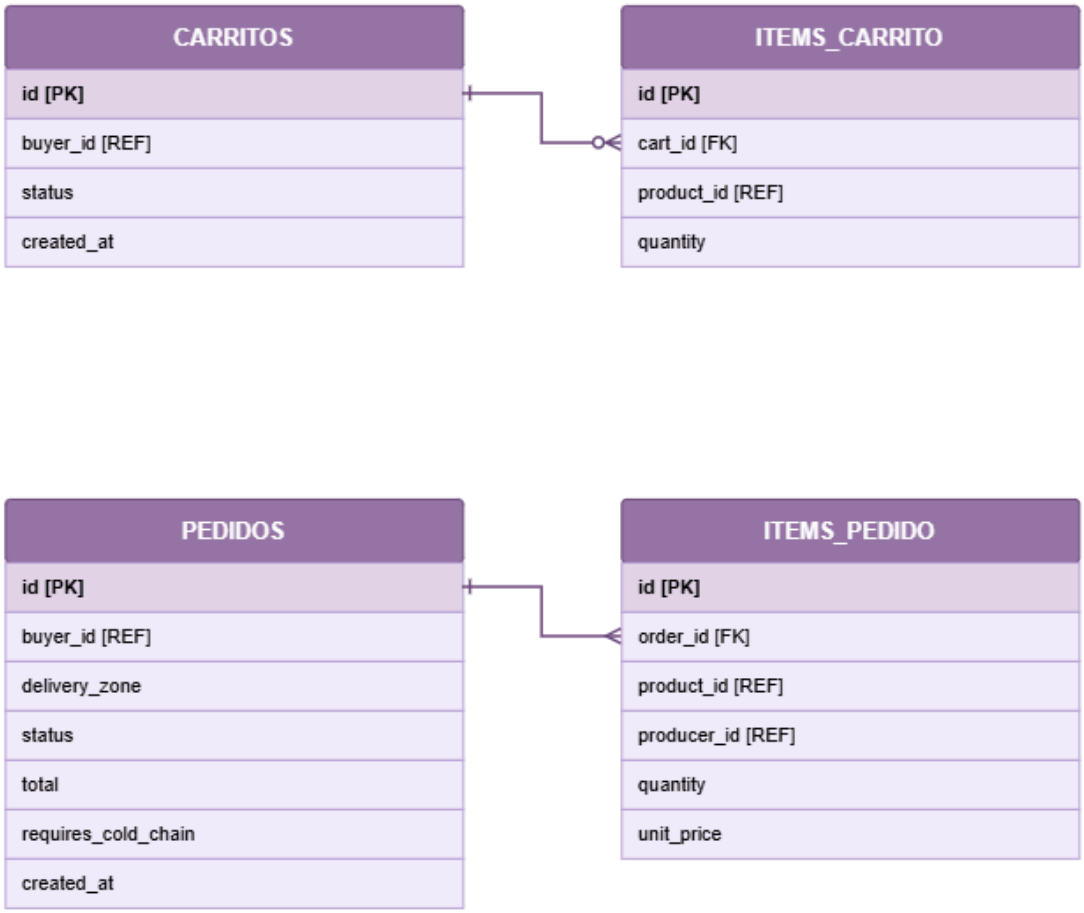
PRODUCTOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador del producto ofrecido.
producer_id	uuid	REF Productores	No	Debe corresponder a un productor activo.
name	varchar(150)	-	No	Texto no vacío.
category_id	uuid	FK CATEGORIAS.id	No	Sustituye category; ON DELETE RESTRICT.
unit	varchar(8)	-	No	DEFAULT 'kg'; CHECK unit = 'kg'.
min_order_quantity	numeric(12,3)	-	No	CHECK > 0; mínimo definido por productor.
base_price	numeric(14,2)	-	No	CHECK > 0; precio de referencia por kg.
low_stock_threshold	numeric(12,3)	-	No	CHECK >= 0; umbral inferior en kg.
high_stock_threshold	numeric(12,3)	-	No	CHECK > low_stock_threshold.
active	boolean	-	No	DEFAULT true.
Índices (producer_id, active) y (category_id, active).				

LOTES_DE_COSECHA

Campo	Tipo	Clave / referencia	Nulo	Regla
batch_id	uuid	PK	No	Identificador canónico compartido como REF.
product_id	uuid	FK PRODUCTOS.id	No	ON DELETE RESTRICT.
farm_id	uuid	REF Productores.FINCAS	No	Validar que la finca pertenece al productor del producto.
harvest_date	date	-	No	Fecha de cosecha.
quantity_kg	numeric(12,3)	-	No	CHECK > 0; cantidad inicial.
expiry_estimate	date	-	No	CHECK >= harvest_date.
status	varchar(12)	-	No	CHECK IN ('registered','published','closed').
Índices (product_id, expiry_estimate) y (farm_id, harvest_date).				

FarmToTable – Modelo de datos: Servicio de Pedidos



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).



Cada fila conecta una PK con su FK local. Una entidad repetida representa la misma tabla. Las referencias REF son externas y no crean FK entre bases.

CARRITOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de carrito.
buyer_id	uuid	REF Identidad	No	Comprador autenticado.
status	varchar(12)	-	No	CHECK IN ('active','converted','abandoned').
created_at	timestamptz	-	No	DEFAULT now().
UNIQUE parcial (buyer_id) WHERE status = 'active'.				

ITEMS_CARRITO

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de línea.
cart_id	uuid	FK CARRITOS.id	No	ON DELETE CASCADE.
product_id	uuid	REF Catálogo	No	Producto consultado por API.
quantity	numeric(12,3)	-	No	CHECK > 0; kg.
UNIQUE(cart_id, product_id).				

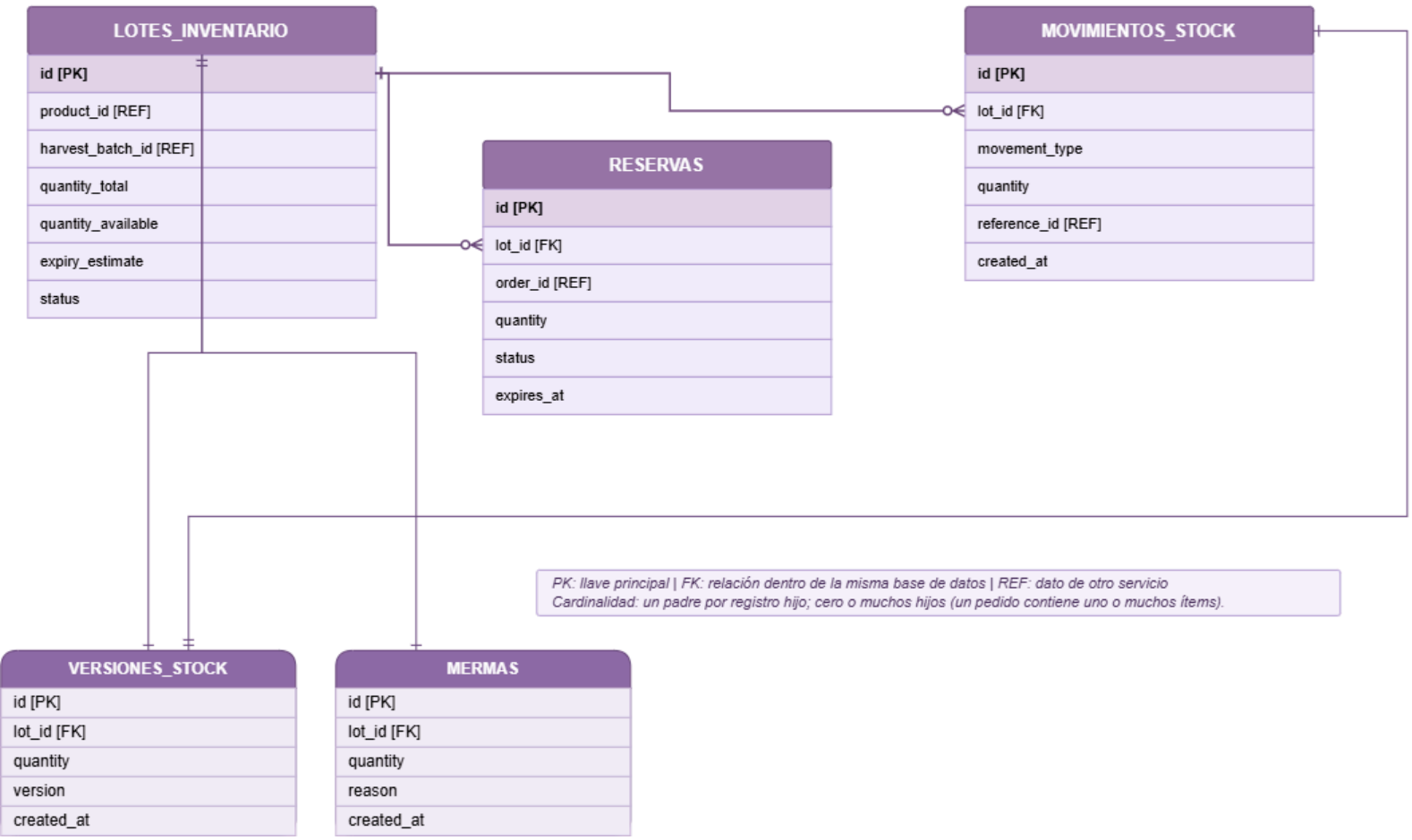
PEDIDOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador del pedido.
buyer_id	uuid	REF Identidad	No	Comprador autenticado.
delivery_zone	varchar(80)	REF Logística.ZONAS.code	No	Zona validada al comprar; instantánea de la agrupación.
delivery_address	text	-	No	Dirección aceptada al comprar.
status	varchar(20)	-	No	CHECK IN ('pending','confirmed','in_transit','delivered','partially_delivered','cancelled').
total	numeric(14,2)	-	No	CHECK >= 0; suma de las líneas en COP.
requires_cold_chain	boolean	-	No	Necesidad de frío del pedido.
created_at	timestamptz	-	No	DEFAULT now().
window_start	timestamptz	-	No	Inicio de ventana aceptada por comprador.
window_end	timestamptz	-	No	CHECK > window_start; compromiso original.
payment_status	varchar(16)	-	No	CHECK IN ('pending','authorized','completed','failed','voided').
fulfillment_closed_at	timestamptz	-	Sí	NULL mientras queden entregas sin resolver; cierre único.
fulfillment_plan_id	uuid	REF Logística.PLANES_ENTREGA	Sí	NULL hasta fulfillment.plan_ready; fija el conjunto de entregas del cierre.
Índices (buyer_id, created_at DESC) y (status, delivery_zone).				

ITEMS_PEDIDO

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Referencia usada por Inventario y Pagos.
order_id	uuid	FK PEDIDOS.id	No	ON DELETE RESTRICT.
product_id	uuid	REF Catálogo	No	Producto comprado.
producer_id	uuid	REF Productores	No	Productor al confirmar la compra.
quantity	numeric(12,3)	-	No	CHECK > 0; kg.
unit_price	numeric(14,2)	-	No	CHECK > 0; COP/kg aceptado.
price_version_id	uuid	REF Catálogo.HISTORIAL_DE_PRECIOS	No	Versión aceptada.
min_quantity_snapshot	numeric(12,3)	-	No	CHECK > 0 y quantity >= min_quantity_snapshot.
UNIQUE(order_id, product_id); índice (producer_id, order_id).				

FarmToTable – Modelo de datos: Servicio de Inventario Perecedero



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

LOTES_INVENTARIO

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador local.
product_id	uuid	REF Catálogo	No	Producto del lote de origen.
harvest_batch_id	uuid	REF Catálogo.LOTES_DE_COSECHA	No	UNIQUE; un registro de inventario por lote.
quantity_total	numeric(12,3)	-	No	CHECK >= 0; saldo físico actual en kg.
quantity_available	numeric(12,3)	-	No	CHECK >= 0 AND <= quantity_total.
expiry_estimate	date	-	No	Copia de fecha de vencimiento.
status	varchar(12)	-	No	CHECK IN ('available','blocked','expired','depleted').
UNIQUE(harvest_batch_id); índice FEFO (product_id, status, expiry_estimate, id).				

RESERVAS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de reserva.
lot_id	uuid	FK LOTES_INVENTARIO.id	No	ON DELETE RESTRICT.
order_id	uuid	REF Pedidos	No	Pedido dueño de la reserva.
order_item_id	uuid	REF Pedidos.ITEMS_PEDIDO	No	Línea atendida con este lote.
quantity	numeric(12,3)	-	No	CHECK > 0; kg.
status	varchar(12)	-	No	CHECK IN ('active','released','consumed','expired').
expires_at	timestamptz	-	No	Límite de la reserva.
UNIQUE(order_item_id, lot_id); índices (order_id, status) y (status, expires_at).				

MOVIMIENTOS_STOCK

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador usado por el evento.
lot_id	uuid	FK LOTES_INVENTARIO.id	No	ON DELETE RESTRICT.
movement_type	varchar(15)	-	No	CHECK IN ('entry','dispatch','waste','return').
quantity	numeric(12,3)	-	No	CHECK > 0; magnitud en kg; el tipo define el signo.
reference_id	uuid	REF según reference_type	Sí	Pedido/entrega de origen si aplica.
reference_type	varchar(12)	-	Sí	CHECK IN ('order','delivery'); ambos campos nulos o ambos presentes.
reason	varchar(20)	-	Sí	Para waste es obligatorio: expiry, damage o cold_chain.
source_operation_id	uuid	-	No	UNIQUE; evita aplicar dos veces la misma operación.
created_at	timestamptz	-	No	DEFAULT now().
UNIQUE(source_operation_id); índices (lot_id, created_at) y (movement_type, created_at).				

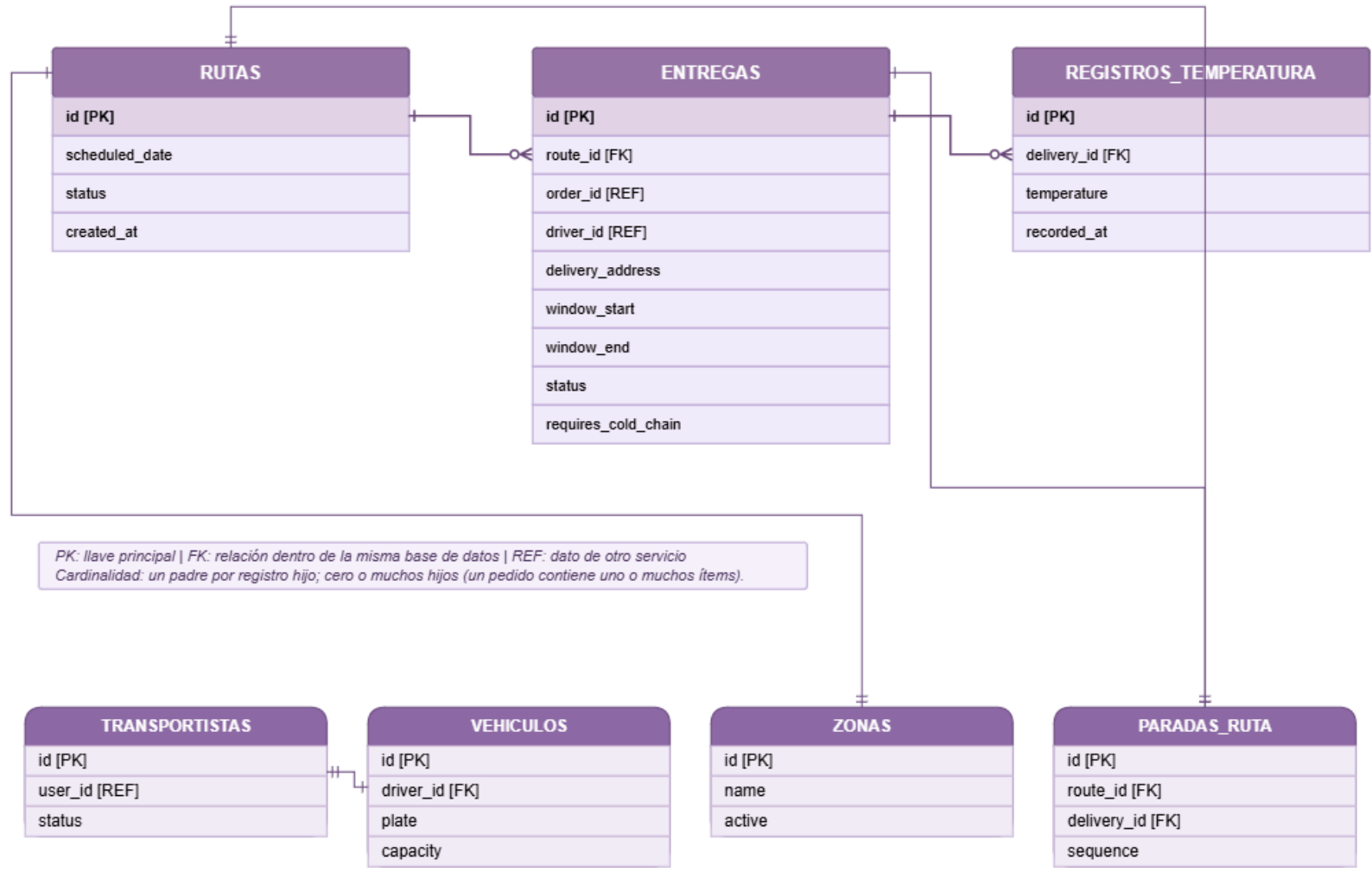
VERSIONES_STOCK

Campo	Tipo	Clave / referencia	Nulo	Regla
product_id	uuid	PK y REF Catálogo	No	Una fila por producto; no FK externa.
stock_version	bigint	-	No	DEFAULT 1; CHECK > 0; aumenta en cada cambio de saldo.
PK(product_id). Se bloquea esta fila antes de modificar lotes del producto y publicar inventory.stock_changed; versión y saldos se confirman juntos.				

Reglas de operación

Las reservas bloquean las filas del lote y no permiten saldo negativo: disponible = físico - SUM(reservas activas). Reservar reduce solo disponible; liberar lo aumenta; despachar reduce físico y consume la reserva. Una merma reduce físico y disponible; si afecta unidades reservadas, antes se reasignan o cancelan sus reservas. POST /inventory/waste registra la merma con usuario autorizado, motivo y operación idempotente; publica inventory.waste_recorded (movement_id, lot_id, harvest_batch_id, product_id, quantity_kg, reason, occurred_at). cold_chain.alert solo solicita inspección: no prueba una pérdida ni se cuenta como merma. Una pérdida en tránsito se registra como return a cuarentena y waste en la misma transacción, sin liberar disponibilidad para venta. delivery.failed por sí solo tampoco vuelve a poner alimentos en venta. inventory.reserved incluye order_id y asignaciones de reservation_id, order_item_id, lot_id, harvest_batch_id, farm_id y quantity. Cada cambio publica inventory.stock_changed con product_id, available_kg y versión creciente por producto.

FarmToTable – Modelo de datos: Servicio de Logística y Rutas



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

TRANSPORTISTAS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador operativo local.
user_id	uuid	REF Identidad	No	UNIQUE; cuenta autenticada.
full_name	varchar(150)	-	No	Nombre operativo no vacío.
phone	varchar(25)	-	No	Contacto operativo.
license_number	varchar(40)	-	No	UNIQUE.
active	boolean	-	No	DEFAULT true.
UNIQUE(user_id); UNIQUE(license_number).				

VEHICULOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador del vehículo.
plate	varchar(12)	-	No	UNIQUE; placa normalizada en mayúsculas.
capacity_kg	numeric(12,3)	-	No	CHECK > 0.
refrigerated	boolean	-	No	DEFAULT false.
active	boolean	-	No	DEFAULT true.
UNIQUE(plate); índice (active, refrigerated).				

RUTAS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de ruta.
driver_id	uuid	FK TRANSPORTISTAS.id	No	ON DELETE RESTRICT.
vehicle_id	uuid	FK VEHICULOS.id	No	ON DELETE RESTRICT.
scheduled_date	timestamptz	-	No	Inicio programado.
scheduled_end	timestamptz	-	No	CHECK > scheduled_date.
status	varchar(12)	-	No	CHECK IN ('planned','in_transit','completed','cancelled').
created_at	timestamptz	-	No	DEFAULT now().
zone_code	varchar(80)	FK ZONAS.code	No	Zona de destinos agrupados; recogidas pueden quedar fuera.
Índices (driver_id, scheduled_date) y (vehicle_id, scheduled_date). Índice (zone_code, scheduled_date).				

ENTREGAS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de entrega parcial o total.
route_id	uuid	FK RUTAS.id	No	ON DELETE RESTRICT.
order_id	uuid	REF Pedidos	No	Pedido asociado; no es UNIQUE.
delivery_address	text	-	No	Instantánea del destino.
window_start	timestamptz	-	No	Inicio de ventana.
window_end	timestamptz	-	No	CHECK > window_start.
status	varchar(12)	-	No	CHECK IN ('scheduled','in_transit','completed','partial','failed','cancelled').
requires_cold_chain	boolean	-	No	Necesidad de frío.
dispatched_at	timestamptz	-	Sí	Obligatorio en in_transit y completed.
delivered_at	timestamptz	-	Sí	Obligatorio en completed/partial; CHECK >= dispatched_at.
closed_at	timestamptz	-	Sí	Fecha de resolución final; obligatoria en estado terminal.
cancelled_at	timestamptz	-	Sí	Obligatorio en cancelled; fecha efectiva, no retroactiva.
plan_id	uuid	FK PLANES_ENTREGA.id	No	La entrega y el plan deben compartir order_id.
Índices (route_id, status) y (order_id). Índice (plan_id).				

ENTREGAS_LOTES

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de asignación.
delivery_id	uuid	FK ENTREGAS.id	No	ON DELETE RESTRICT.
reservation_id	uuid	REF Inventario.RESERVAS	No	Reserva física asignada.
harvest_batch_id	uuid	REF Catálogo.LOTES_DE_COSECHA	No	Lote canónico de origen.
farm_id	uuid	REF Productores.FINCAS	No	Finca validada del lote.
quantity_kg	numeric(12,3)	-	No	CHECK > 0.
pickup_address	text	-	No	Instantánea de dirección de recogida.
pickup_latitude	numeric(9,6)	-	No	CHECK entre -90 y 90.
pickup_longitude	numeric(9,6)	-	No	CHECK entre -180 y 180.
accepted_quantity_kg	numeric(12,3)	-	Sí	NULL pendiente; al cerrar, valor entre 0 y quantity_kg aceptado por comprador.
UNIQUE(delivery_id, reservation_id); índices (reservation_id) y (harvest_batch_id).				

REGISTROS_TEMPERATURA

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de medición.
delivery_id	uuid	FK ENTREGAS.id	No	ON DELETE RESTRICT.
temperature	numeric(5,2)	-	No	Grados Celsius, puede ser negativa.
recorded_at	timestamptz	-	No	Instante de medición.
sensor_event_id	varchar(100)	-	No	UNIQUE; deduplicación de lecturas.
UNIQUE(sensor_event_id); índice (delivery_id, recorded_at).				

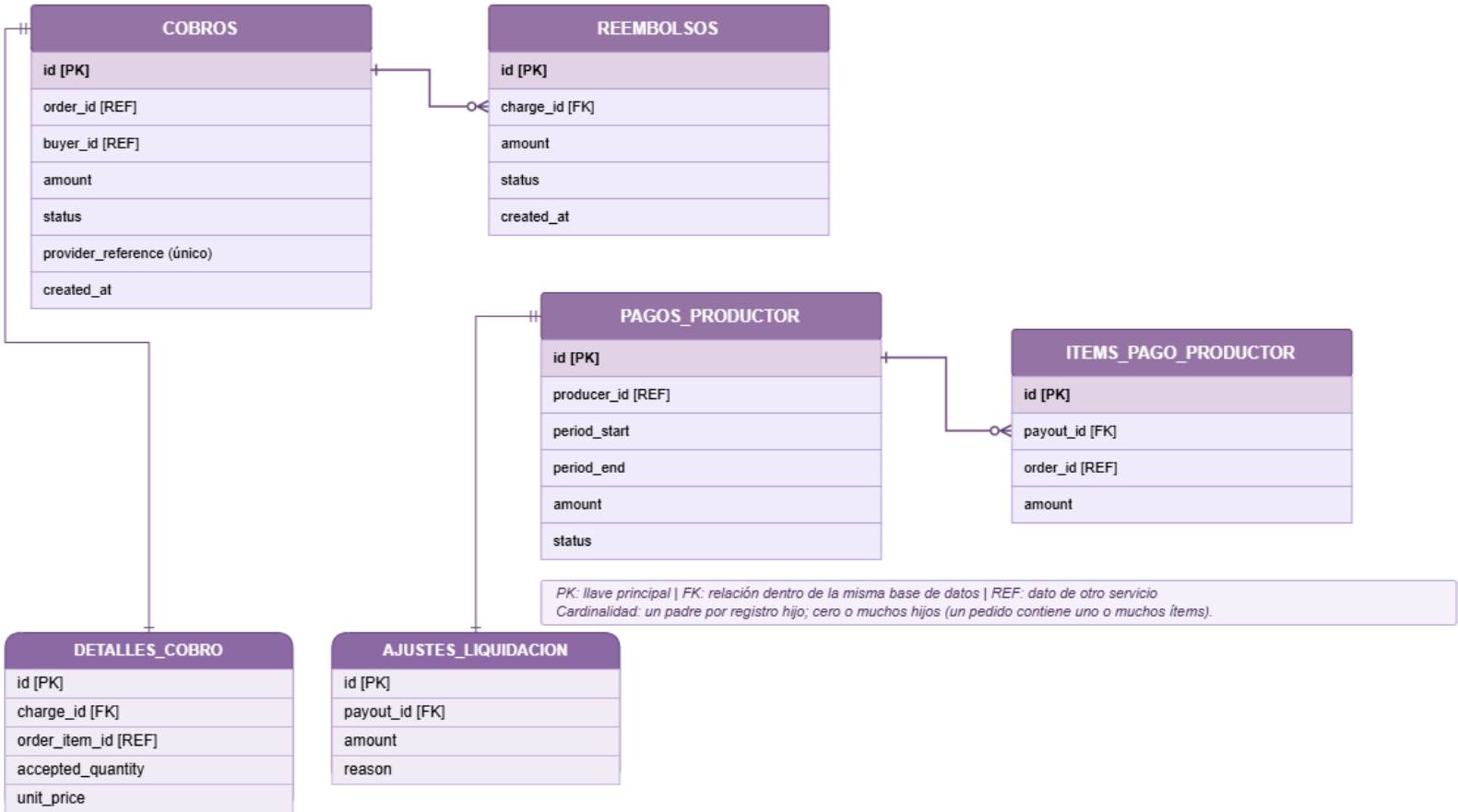
ZONAS

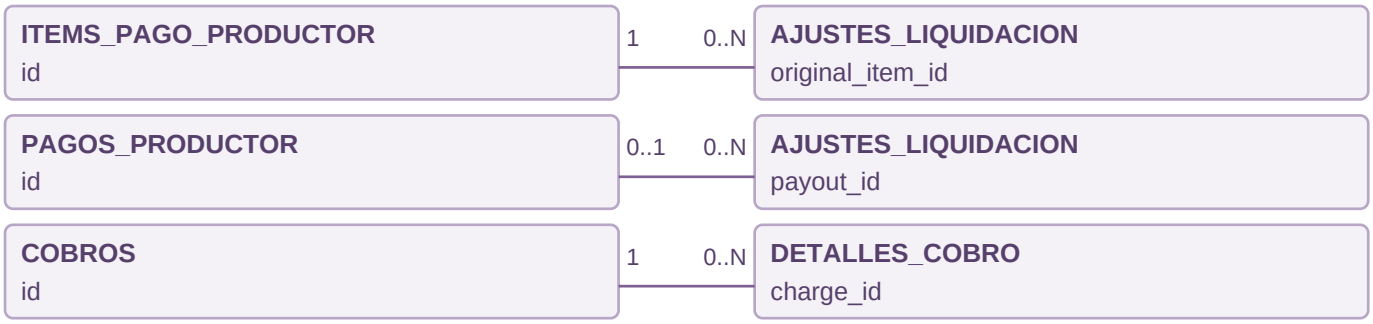
Campo	Tipo	Clave / referencia	Nulo	Regla
code	varchar(80)	PK	No	Código estable de zona operativa.
name	varchar(120)	-	No	Nombre no vacío.
active	boolean	-	No	DEFAULT true; baja lógica.
PK(code). El administrador logístico define las zonas y verifica la asignación de direcciones antes de confirmar la entrega.				

PARADAS_RUTA

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de parada planificada.
route_id	uuid	FK RUTAS.id	No	Ruta propietaria.
delivery_id	uuid	FK ENTREGAS.id	No	Entrega atendida; debe pertenecer a la misma ruta.
sequence	integer	-	No	CHECK > 0; orden de visita.
type	varchar(12)	-	No	CHECK IN ('pickup','dropoff').
farm_id	uuid	REF Productores.FINCAS	Sí	Obligatorio en pickup y NULL en dropoff.
address	text	-	No	Instantánea de la dirección planificada.
planned_at	timestamptz	-	No	Hora estimada de llegada.
actual_at	timestamptz	-	Sí	Hora real, NULL antes de la visita.
UNIQUE(route_id, sequence); índice (delivery_id). La aplicación exige recogida anterior a entrega, secuencia factible y pertenencia de farm_id a los lotes asignados.				

FarmToTable – Modelo de datos: Servicio de Pagos





Cada fila conecta una PK con su FK local. Una entidad repetida representa la misma tabla. Las referencias REF son externas y no crean FK entre bases.

COBROS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador del intento de cobro.
order_id	uuid	REF Pedidos	No	Pedido; admite intentos fallidos sucesivos.
buyer_id	uuid	REF Identidad	No	Comprador del pedido.
amount	numeric(14,2)	-	Sí	NULL antes de captura; CHECK >= 0 AND <= authorized_amount; bruto efectivamente cobrado.
status	varchar(16)	-	No	CHECK IN ('pending','authorized','completed','failed','refunded','voided').
provider_reference	varchar(120)	-	Sí	UNIQUE; NULL antes de respuesta de pasarela.
idempotency_key	uuid	-	No	UNIQUE; clave por intento.
created_at	timestamptz	-	No	DEFAULT now().
authorized_amount	numeric(14,2)	-	No	CHECK > 0; total del pedido cuya autorización se solicita.
authorization_expires_at	timestamptz	-	Sí	Fecha devuelta por pasarela; requerida al autorizar.
UNIQUE(provider_reference); UNIQUE(idempotency_key); índice (order_id). UNIQUE parcial (order_id) WHERE status IN ('pending','authorized','completed','refunded').				

REEMBOLSOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de reembolso por línea.
charge_id	uuid	FK COBROS.id	No	ON DELETE RESTRICT.
order_item_id	uuid	REF Pedidos.ITEMS_PEDIDO	No	Línea que se devuelve.
producer_id	uuid	REF Productores	No	Productor de la línea.
amount	numeric(14,2)	-	No	CHECK > 0; bruto devuelto.
status	varchar(12)	-	No	CHECK IN ('pending','completed','failed').
provider_reference	varchar(120)	-	Sí	UNIQUE si existe.
idempotency_key	uuid	-	No	UNIQUE.
created_at	timestamptz	-	No	DEFAULT now().
UNIQUE(provider_reference); UNIQUE(idempotency_key); índices (charge_id, status) y (order_item_id, status).				

PAGOS_PRODUCTOR

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de liquidación.
producer_id	uuid	REF Productores	No	Productor beneficiario.
period_start	date	-	No	Inicio inclusivo del periodo.
period_end	date	-	No	Fin exclusivo; CHECK > period_start.
gross_amount	numeric(14,2)	-	No	CHECK >= 0; suma del bruto de los detalles.
refund_amount	numeric(14,2)	-	No	CHECK >= 0; devoluciones antes de liquidar.
commission_amount	numeric(14,2)	-	No	CHECK >= 0; suma de comisiones.
adjustment_amount	numeric(14,2)	-	No	DEFAULT 0; suma firmada de ajustes posteriores.
net_amount	numeric(14,2)	-	No	CHECK = gross_amount - refund_amount - commission_amount + adjustment_amount.
status	varchar(12)	-	No	CHECK IN ('draft','scheduled','paid','failed','carried').
UNIQUE(producer_id, period_start, period_end); índice (producer_id, status). amount se sustituye por net_amount.				

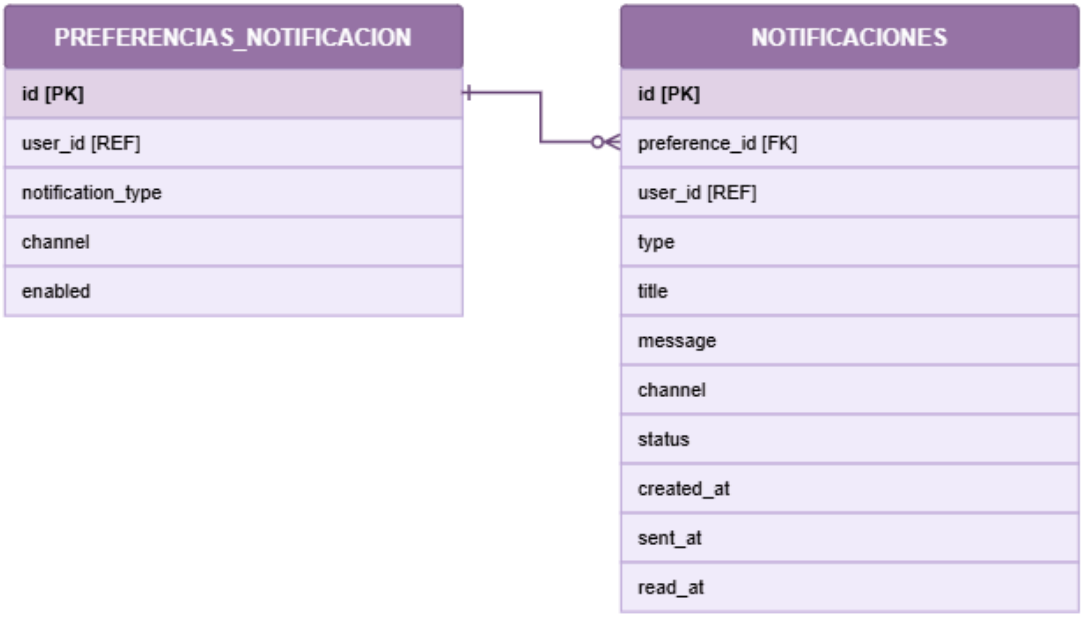
ITEMS_PAGO_PRODUCTOR

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de detalle.
payout_id	uuid	FK PAGOS_PRODUCTOR.id	No	ON DELETE RESTRICT.
charge_id	uuid	FK COBROS.id	No	Cobro capturado que respalda el ingreso.
order_id	uuid	REF Pedidos	No	Pedido de origen.
order_item_id	uuid	REF Pedidos.ITEMS_PEDIDO	No	UNIQUE; no liquidar dos veces una línea.
gross_amount	numeric(14,2)	-	No	CHECK > 0; copia de DETALLES_COBRO.gross_amount capturado.
refund_amount	numeric(14,2)	-	No	CHECK entre 0 y gross_amount.
commission_rate	numeric(7,6)	-	No	CHECK entre 0 y 1; copia de la tasa pactada en DETALLES_COBRO.
commission_amount	numeric(14,2)	-	No	CHECK = round((gross_amount - refund_amount) × commission_rate, 2).
net_amount	numeric(14,2)	-	No	CHECK = gross_amount - refund_amount - commission_amount.
charge_detail_id	uuid	FK DETALLES_COBRO.id	No	UNIQUE; fuente de bruto y tasa; debe corresponder al mismo charge_id.
UNIQUE(order_item_id); índices (payout_id) y (charge_id). amount se sustituye por net_amount. UNIQUE(charge_detail_id).				

AJUSTES_LIQUIDACION

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de ajuste posterior.
refund_id	uuid	FK REEMBOLSOS.id	No	UNIQUE; devolución ya completada.
original_item_id	uuid	FK ITEMS_PAGO_PRODUCTOR.id	No	Detalle original ya cerrado.
payout_id	uuid	FK PAGOS_PRODUCTOR.id	Sí	NULL hasta asignar el ajuste a un nuevo periodo.
amount	numeric(14,2)	-	No	Importe firmado: negativo al recuperar dinero del productor.
created_at	timestamptz	-	No	DEFAULT now().
UNIQUE(refund_id); índices (original_item_id) y (payout_id).				

FarmToTable – Modelo de datos: Servicio de Notificaciones



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

PREFERENCIAS_NOTIFICACION

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de preferencia.
user_id	uuid	REF Identidad	No	Usuario propietario.
notification_type	varchar(60)	-	No	Tipo de evento permitido por catálogo de la aplicación.
channel	varchar(10)	-	No	CHECK IN ('email','sms','in_app').
enabled	boolean	-	No	DEFAULT true.
UNIQUE(user_id, notification_type, channel).				

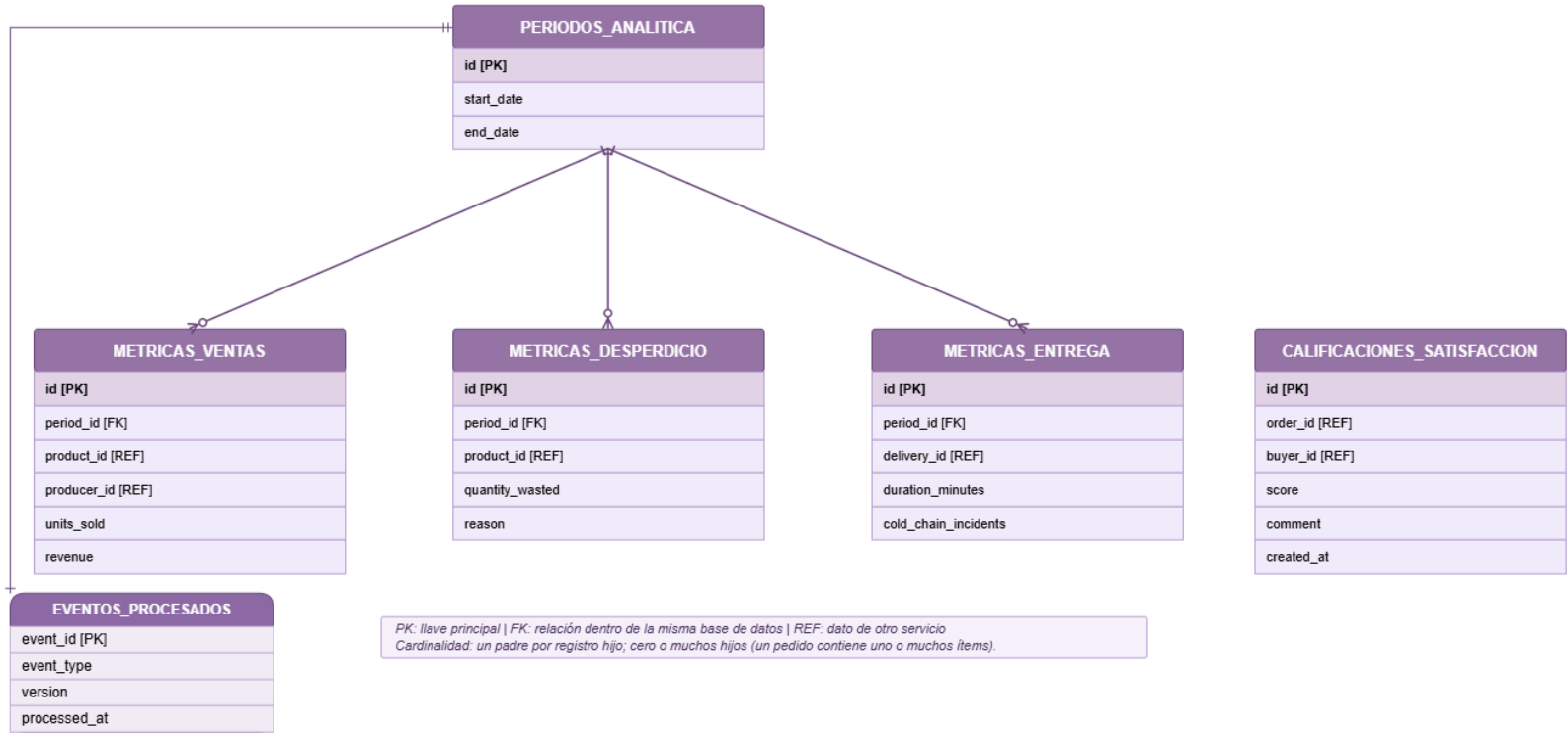
NOTIFICACIONES

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de notificación.
preference_id	uuid	FK PREFERENCIAS_NOTIFICACION.id	Sí	ON DELETE SET NULL; puede ser aviso obligatorio.
user_id	uuid	REF Identidad	No	Destinatario al generar el aviso.
type	varchar(60)	-	No	Tipo de evento reconocido por la aplicación.
title	varchar(160)	-	No	Título no vacío.
message	text	-	No	Contenido del aviso.
channel	varchar(10)	-	No	CHECK IN ('email','sms','in_app').
status	varchar(12)	-	No	CHECK IN ('pending','sent','failed','read').
source_event_id	uuid	-	No	Identificador del evento que originó el aviso.
created_at	timestamptz	-	No	DEFAULT now().
sent_at	timestamptz	-	Sí	NULL antes del envío; CHECK >= created_at.
read_at	timestamptz	-	Sí	NULL hasta lectura; exige sent_at y CHECK >= sent_at.
UNIQUE(source_event_id, user_id, channel, type); índices (user_id, created_at DESC) y (status, created_at).				

Reglas de operación

Al registrar un aviso con preference_id se valida que coincidan usuario, tipo y canal. status sent o read exige sent_at; status read exige read_at. La entrega se reintenta con la misma clave idempotente; no crea otra fila ni otro envío si el proveedor soporta idempotencia.

FarmToTable – Modelo de datos: Servicio de Analítica



PK: llave principal | FK: relación dentro de la misma base de datos | REF: dato de otro servicio
Cardinalidad: un padre por registro hijo; cero o muchos hijos (un pedido contiene uno o muchos ítems).

METRICAS_VENTAS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de agregado.
period_id	uuid	FK PERIODOS_ANALITICA.id	No	ON DELETE RESTRICT.
product_id	uuid	REF Catálogo	No	Producto agregado.
producer_id	uuid	REF Productores	No	Productor de la venta.
units_sold	numeric(14,3)	-	No	CHECK >= 0; kg vendidos de pedidos cobrados.
revenue	numeric(16,2)	-	No	CHECK >= 0; ingreso bruto capturado en COP.
UNIQUE(period_id, product_id, producer_id). Las devoluciones se reportan por separado desde EVENTOS_PROCESADOS; no se restan dos veces.				

METRICAS_DESPERDICIO

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador de agregado de merma.
period_id	uuid	FK PERIODOS_ANALITICA.id	No	ON DELETE RESTRICT.
product_id	uuid	REF Catálogo	No	Producto afectado.
quantity_wasted	numeric(14,3)	-	No	CHECK >= 0; kg confirmados como merma.
reason	varchar(20)	-	No	CHECK IN ('expiry','damage','cold_chain').
UNIQUE(period_id, product_id, reason).				

METRICAS_ENTREGA

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador del hecho de entrega.
period_id	uuid	FK PERIODOS_ANALITICA.id	No	Periodo UTC en el que vence window_end; no se mueve al llegar tarde.
delivery_id	uuid	REF Logística	No	UNIQUE.
duration_minutes	numeric(12,2)	-	Sí	NULL sin entrega efectiva; CHECK >= 0 cuando exista.
cold_chain_incidents	integer	-	No	CHECK >= 0; alertas distintas, no mermas.
window_start	timestamptz	-	No	Inicio comprometido; copia de Logística.
window_end	timestamptz	-	No	CHECK > window_start; determina el periodo de vencimiento.
delivered_at	timestamptz	-	Sí	NULL si no hubo entrega; dato de la fuente.
status	varchar(12)	-	No	CHECK IN ('scheduled','in_transit','completed','partial','failed','cancelled').
cancelled_at	timestamptz	-	Sí	Fecha de cancelación; permite distinguir cancelación antes o después de la ventana.
UNIQUE(delivery_id); índice (period_id).				

CALIFICACIONES_SATISFACCION

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK y REF Pedidos.VALORACIONES	No	Mismo rating_id de la fuente.
period_id	uuid	FK PERIODOS_ANALITICA.id	No	Periodo de created_at.
order_id	uuid	REF Pedidos	No	UNIQUE; una valoración por pedido.
buyer_id	uuid	REF Identidad	No	Comprador de la valoración.
score	smallint	-	No	CHECK BETWEEN 1 AND 5.
comment	varchar(1000)	-	Sí	Copia opcional del comentario.
created_at	timestamptz	-	No	Fecha de la valoración original.
UNIQUE(order_id); índice (period_id, score).				

EVENTOS_PROCESADOS

Campo	Tipo	Clave / referencia	Nulo	Regla
event_id	uuid	PK	No	Identificador global del evento recibido.
event_type	varchar(80)	-	No	Tipo reconocido y versionado.
source_service	varchar(40)	-	No	Servicio publicador.
entity_id	uuid	-	No	Identificador del hecho de origen.
entity_version	bigint	-	No	CHECK > 0; versión del hecho.
occurred_at	timestamptz	-	No	Fecha efectiva del evento.
processed_at	timestamptz	-	No	DEFAULT now().
payload	jsonb	-	No	CHECK jsonb_typeof(payload) = 'object'; contrato validado por consumidor.
schema_version	integer	-	No	CHECK > 0; versión del contrato.
correlation_id	uuid	-	No	Identificador del flujo distribuido.
UNIQUE(source_service, event_type, entity_id, entity_version); índices (event_type, occurred_at) y (entity_id). JSONB solo conserva el sobre de auditoría; las métricas permanecen en tablas relacionales tipadas.				

Reglas de operación

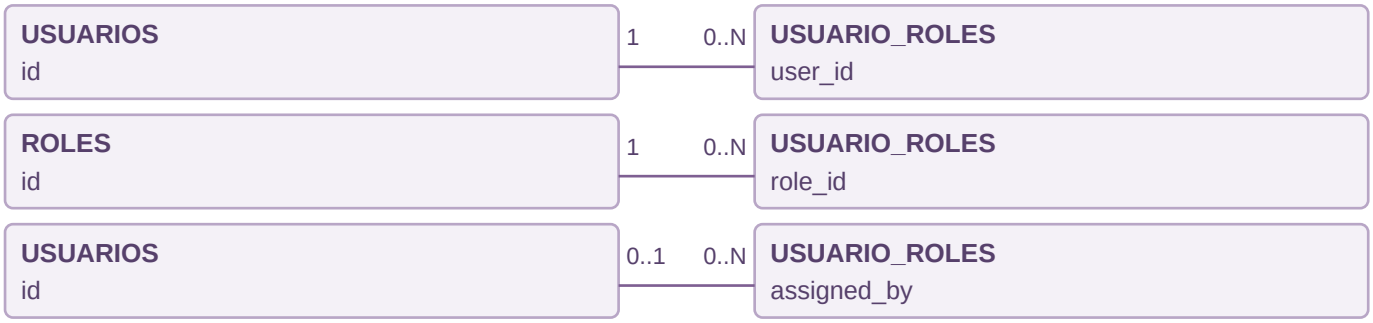
METRICAS_VENTAS suma kg aceptados e importe capturado de payment.completed; no cuenta order.created como venta. payment.refunded permite reportar devoluciones por separado. Mermas llegan exclusivamente de inventory.waste_recorded. delivery.scheduled crea la cohorte de ventanas comprometidas y los eventos posteriores actualizan su cumplimiento. Los agregados se actualizan junto con EVENTOS_PROCESADOS, deduplicando event_id y versión. Los datos tardíos corrigen el periodo de negocio correspondiente. Los indicadores y sus denominadores se definen en 5.6.

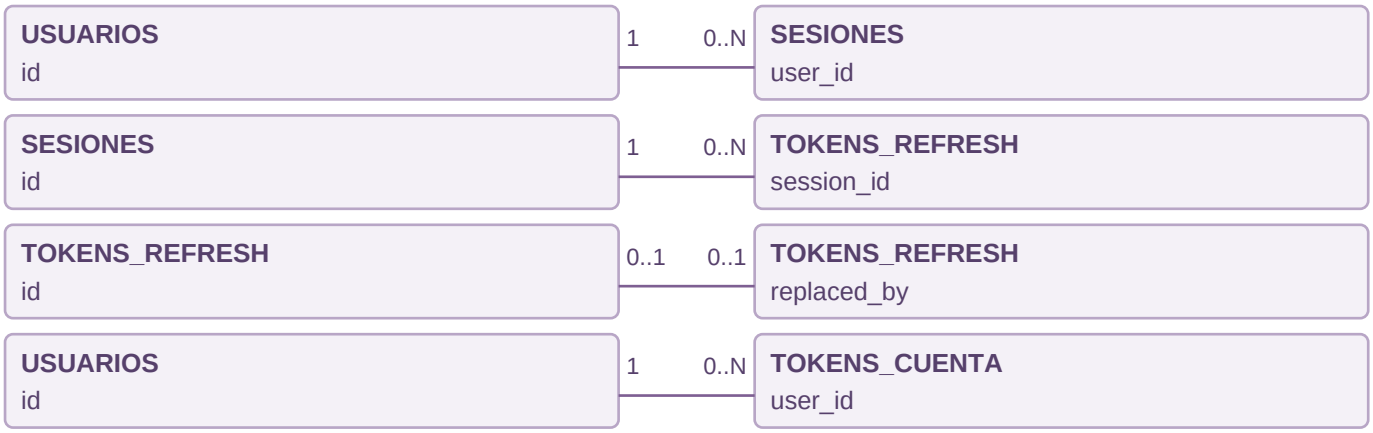
4.9. Identidad y Acceso

Base: identity_db. Identidad es el dueño de las cuentas y de sus sesiones. Ningún otro servicio guarda password_hash ni refresh tokens. El modelo separa roles, sesiones por dispositivo y tokens de un solo uso.

3FN: USUARIOS contiene la cuenta; ROLES y USUARIO_ROLES resuelven la relación N a N; SESIONES agrupa el acceso por dispositivo; TOKENS_REFRESH conserva sus rotaciones. TOKENS_CUENTA separa verificación y recuperación. roles_version es un contador deliberado de invalidación, no otra fuente de permisos; los roles vigentes se consultan en USUARIO_ROLES. Los hashes conservan evidencia verificable de tokens sin guardar su valor utilizable.

Relaciones del servicio





Cada fila conecta una PK con su FK local. Una entidad repetida representa la misma tabla. Las referencias REF son externas y no crean FK entre bases.

USUARIOS

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador global; claim sub del JWT.
email	varchar(254)	-	No	UNIQUE; minúsculas y sin espacios externos.
password_hash	varchar(255)	-	Sí	Hash Argon2id codificado con salt y parámetros. NULL solo mientras pending_invite.
status	varchar(20)	-	No	CHECK IN ('pending_email','pending_invite','active','disabled').
email_verified_at	timestamptz	-	Sí	NULL antes de verificar; obligatorio para active.
roles_version	bigint	-	No	DEFAULT 1; CHECK > 0; aumenta al cambiar roles.
created_at	timestamptz	-	No	DEFAULT now().
updated_at	timestamptz	-	No	CHECK >= created_at.
UNIQUE(email). CHECK status = 'pending_invite' OR password_hash IS NOT NULL; CHECK status <> 'active' OR email_verified_at IS NOT NULL. Índice (status).				

ROLES

Campo	Tipo	Clave / referencia	Nulo	Regla
id	smallint	PK	No	Identificador de catálogo fijo.
code	varchar(24)	-	No	UNIQUE; CHECK IN ('buyer','producer','driver','logistics_admin','admin').
UNIQUE(code). No se crean roles arbitrarios mediante registro público.				

USUARIO_ROLES

Campo	Tipo	Clave / referencia	Nulo	Regla
user_id	uuid	PK compuesta; FK USUARIOS.id	No	Cuenta beneficiaria.
role_id	smallint	PK compuesta; FK ROLES.id	No	Rol asignado.
assigned_at	timestamptz	-	No	DEFAULT now().
assigned_by	uuid	FK USUARIOS.id	Sí	Administrador que asignó; NULL solo registro buyer o bootstrap auditado.
PRIMARY KEY(user_id, role_id); índice (role_id, user_id). FK ON DELETE RESTRICT. La API impide retirar o desactivar al último admin activo bajo bloqueo transaccional.				

SESIONES

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Claim sid; familia de refresh tokens de un login.
user_id	uuid	FK USUARIOS.id	No	ON DELETE RESTRICT.
created_at	timestamptz	-	No	DEFAULT now().
expires_at	timestamptz	-	No	CHECK > created_at; límite absoluto de sesión.
revoked_at	timestamptz	-	Sí	NULL mientras no esté revocada.
revoke_reason	varchar(30)	-	Sí	Ambos campos de revocación nulos o presentes.
device_label	varchar(120)	-	Sí	Etiqueta informativa del dispositivo; no prueba de identidad.
Índices (user_id, revoked_at) y (expires_at). Sesión válida: no revocada y now() < expires_at.				

TOKENS_REFRESH

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador interno del token.
session_id	uuid	FK SESIONES.id	No	ON DELETE RESTRICT.
token_hash	char(64)	-	No	UNIQUE; SHA-256 del token aleatorio de 256 bits.
created_at	timestamptz	-	No	DEFAULT now().
expires_at	timestamptz	-	No	CHECK > created_at; no excede el límite de sesión.
consumed_at	timestamptz	-	Sí	NULL hasta uso; marca el token como consumido.
replaced_by	uuid	FK TOKENS_REFRESH.id	Sí	UNIQUE; token sucesor de la misma sesión.
UNIQUE(token_hash); índice (session_id, created_at). UNIQUE parcial (session_id) WHERE consumed_at IS NULL. Rotar bloquea sesión y token; consume el anterior antes de insertar el nuevo. Ambos cambios se confirman juntos.				

TOKENS_CUENTA

Campo	Tipo	Clave / referencia	Nulo	Regla
id	uuid	PK	No	Identificador interno de verificación o recuperación.
user_id	uuid	FK USUARIOS.id	No	ON DELETE RESTRICT.
purpose	varchar(20)	-	No	CHECK IN ('verify_email','reset_password','invite').
token_hash	char(64)	-	No	UNIQUE; SHA-256 de secreto aleatorio de 256 bits.
created_at	timestamptz	-	No	DEFAULT now().
expires_at	timestamptz	-	No	CHECK > created_at.
used_at	timestamptz	-	Sí	NULL hasta consumo o invalidación por reemplazo.
UNIQUE(token_hash); índices (user_id, purpose) y (expires_at). El consumo exige propósito correcto, token vigente y used_at NULL, bajo bloqueo de fila.				

4.10. Persistencia de eventos

Cada publicador guarda OUTBOX_EVENTOS en la misma transacción que el cambio local. Un worker publica en RabbitMQ y reintenta hasta recibir confirmación. El consumidor confirma el mensaje después de guardar el efecto y su deduplicación. Analítica usa EVENTOS_PROCESADOS; los demás consumidores usan INBOX_EVENTOS. Los tokens y contraseñas nunca entran en el payload.

OUTBOX_EVENTOS

Campo	Tipo	Clave	Nulo	Regla
event_id	uuid	PK	No	Identificador global único.
event_type	varchar(80)	-	No	Contrato del evento.
entity_id	uuid	-	No	Entidad propietaria del cambio.
entity_version	bigint	-	No	CHECK > 0; secuencia por entidad.
schema_version	integer	-	No	CHECK > 0.
correlation_id	uuid	-	No	Flujo distribuido.
occurred_at	timestamptz	-	No	Fecha efectiva del hecho.
payload	jsonb	-	No	CHECK jsonb_typeof(payload) = 'object'; contrato tipado validado.
published_at	timestamptz	-	Sí	NULL hasta confirmación del broker.

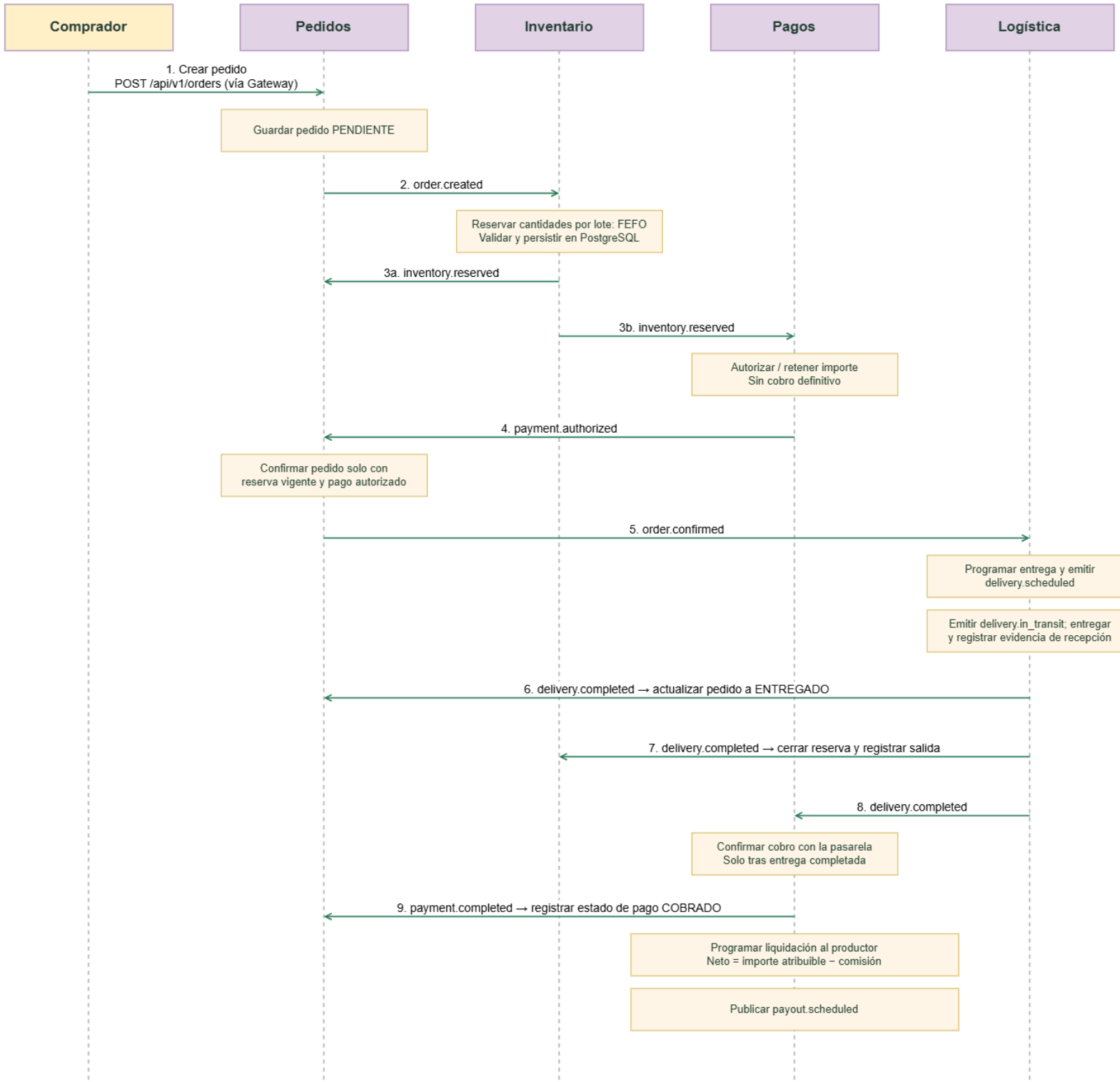
INBOX_EVENTOS

Campo	Tipo	Clave	Nulo	Regla
event_id	uuid	PK	No	Evento procesado una sola vez.
processed_at	timestamptz	-	No	DEFAULT now().

OUTBOX: índice parcial (occurred_at) WHERE published_at IS NULL. INBOX: PK(event_id). Las filas técnicas se retienen al menos durante el horizonte completo de reintentos; las operaciones de negocio

FarmToTable | Pedido, pago y entrega

Secuencia principal. Las flechas entre servicios representan eventos transportados por RabbitMQ.
El API Gateway intermedia la solicitud REST del comprador; se omite como línea de vida para facilitar la lectura.



Notificaciones consume order.confirmed, delivery.scheduled, delivery.completed, payment.completed y payout.scheduled.
Rutas excluyentes de fallo: pestañas 2, 3 y 4 / anexo A del proyecto. FEFO: primero vence, primero sale; Redis solo sirve como caché de consulta.

5.2. Compensaciones y casos que no son equivalentes

Situación	Decisión y datos afectados
Falta inventario o falla autorización antes de salir	Pedidos cancela. Inventario libera solo reservas activas. Pagos anula la autorización si llegó a existir. Si las respuestas llegan tarde, los consumidores revisan la cancelación antes de actuar.
La autorización no cubre la ventana de entrega	No despachar. Pagos valida authorization_expires_at y el tiempo previsto; si requiere nueva autorización se solicita al comprador antes de continuar.
Entrega parcial o fallida	Cada asignación registra accepted_quantity_kg. Logística cierra completed/partial/failed y no elimina la entrega. Al terminar todo el plan, Pedidos captura el cierre parcial: delivered, partially_delivered o cancelled si no se aceptó nada.
Falla la captura después de entregar	El estado físico no se revierte. payment_status = failed; Pagos concilia con pasarela y reintenta idempotentemente o solicita regularización. No libera producto ya despachado.
Cancelación o rechazo tras recoger producto	Las reservas consumidas no vuelven a disponibles. El retorno se inspecciona y registra; si hay pérdida, se reconoce merma. Solo producto realmente recibido y apto puede volver a stock.
Reembolso tras haber cobrado	Pagos registra REEMBOLSOS y ajusta la liquidación; nunca resta dos veces el mismo refund_id.
Respuesta de pasarela ambigua	Se consulta provider_reference antes de reintentar. No se interpreta un timeout como cobro fallido ni se crea otro intento sin conciliación.

Se aplica timeout propuesto de 3 segundos a consultas internas y hasta tres reintentos con espera creciente solo a operaciones idempotentes. Un circuit breaker limita llamadas a una dependencia que falla. Los eventos no procesables se envían a una cola de revisión; reinsertarlos conserva event_id. El sistema alerta si una Saga queda pendiente y no la marca como completada solo porque venció un temporizador.

5.3. Contratos de eventos que conectan los modelos

El sobre incluye event_id, event_type, source_service, entity_id, entity_version, schema_version, occurred_at y correlation_id. El payload agrega los datos de la tabla. Si una entrega final llega antes de fulfillment.plan_ready, Pedidos difiere su procesamiento y reintenta hasta conocer el plan; no cierra un conjunto incompleto. El consumidor valida versión y tipos; conserva el orden por entidad o descarta versiones antiguas, sin asumir orden global de RabbitMQ.

Evento y dueño	Consumidor y contenido mínimo
harvest.registered · Catálogo	Inventario y Analítica: batch_id, product_id, producer_id, farm_id, harvest_date, quantity_kg y expiry_estimate. Fuente de origen y denominador de merma por cohorte.
inventory.reserved · Inventario	Pedidos y Logística: order_id, reservas con reservation_id, order_item_id, lot_id, harvest_batch_id, farm_id, quantity y expires_at.
payment.authorized/failed · Pagos	Pedidos: charge_id, order_id, etapa (authorization/capture), importe y vencimiento si autoriza. El fallo distingue en qué etapa ocurrió.
fulfillment.plan_ready · Logística	Pedidos: plan_id, order_id y lista definitiva de delivery_id. No se cierra por una entrega aislada.
delivery.scheduled · Logística	Analítica, Pedidos y Notificaciones: plan_id, delivery_id, order_id, window_start, window_end, route_id y cantidad prevista.
delivery.in_transit · Logística	Inventario, Pedidos y Analítica: delivery_id, order_id, dispatched_at y reservas/cantidades despachadas. Consume reservas y registra dispatch.

Evento y dueño	Consumidor y contenido mínimo
delivery.completed/partial/failed/cancelled · Logística	Pedidos y Analítica: delivery_id, plan_id, order_id, estado, ventanas, dispatched_at, delivered_at, closed_at, cancelled_at cuando aplique y kg aceptados por order_item_id/lote.
order.fulfillment_closed · Pedidos	Pagos: order_id, plan_id, fecha de cierre y kg pedidos/aceptados por línea. Es inmutable e idempotente; no supera lo reservado/despachado.
payment.completed · Pagos	Pedidos, Analítica y Notificaciones: charge_id, order_id, amount y líneas con product_id, producer_id, accepted_quantity, unit_price y gross_amount capturado.
payment.refunded; payout.scheduled/completed · Pagos	Analítica y Notificaciones: refund_id o payout_id, referencias y desglose de bruto, devolución, comisión y neto. Cada evento conserva el identificador del hecho.
inventory.stock_changed · Inventario	Catálogo y Analítica: product_id, stock_version, available_kg y saldos físicos/disponibles de lotes afectados. Redis invalida o refresca su copia; no decide reservas.
inventory.movement_recorded; inventory.waste_recorded · Inventario	Analítica: movement_id, lote, producto, tipo, kg y fecha. waste_recorded agrega motivo; es la única fuente del numerador de merma.
cold_chain.alert · Logística	Inventario, Analítica y Notificaciones: alert_id, delivery_id, lotes, lectura y umbral. Abre inspección; no genera por sí misma kg de desperdicio.
rating.created · Pedidos	Analítica: rating_id, order_id, buyer_id, score, comment y created_at. Solo después de una entrega completa según este alcance.

5.4. Consolidación por zona y ventanas

Logística administra ZONAS y valida la zona de la dirección; Pedidos conserva delivery_zone y la ventana que el comprador aceptó. Se propone consolidar cada cinco minutos pedidos confirmados de la misma zona, con ventanas compatibles, vehículo apto para frío y capacidad suficiente. La zona agrupa destinos; la ruta puede recoger en varias fincas fuera de ella.

El planificador ordena PARADAS_RUTA: primero recoge cada lote y después lo entrega. Estima desplazamiento y tiempo de carga/descarga, y verifica llegada dentro de cada ventana. No une las horas prometidas en una ventana amplia que oculte incumplimientos. Si no hay ruta factible, propone otra ventana antes de confirmarla o asigna otra ruta. La decisión inicial puede ser una heurística de proximidad revisada por el administrador; no se promete una optimización matemática global.

Ejemplo: dos pedidos de la zona Norte, de 12 kg y 18 kg, con ventanas 9:00-10:00 y 9:30-10:30, pueden compartir un vehículo de 50 kg si la secuencia permite llegar a ambos a tiempo. Compartir zona no basta: si el recorrido impide cumplir la segunda ventana, se separan. Cada pedido conserva sus líneas, comprador, autorización y liquidación; consolidar rutas no fusiona cobros.

Se bloquean los recursos al asignar: conductor y vehículo no pueden tener rutas superpuestas. La carga máxima se verifica por tramo, no solo sumando ventas. Tras confirmar el plan, la lista de entregas queda fija; una reprogramación conserva el compromiso original para medir el servicio y registra la nueva hora prevista en la parada. No se reescribe window_end después de una llegada tardía.

5.5. App de transportistas en React Native

La app móvil permite iniciar sesión, ver las rutas asignadas, consultar fincas y paradas, registrar recogida/salida, capturar temperatura y cerrar entregas con kg aceptados o motivo de fallo. Consume el Gateway HTTPS; no se conecta a PostgreSQL, Redis ni RabbitMQ. Identidad emite sus tokens y Logística comprueba TRANSPORTISTAS.user_id = sub y la asignación de cada ruta.

El mapa o navegación ayuda al conductor, pero la autoridad sobre secuencia, cantidades y estado sigue siendo Logística. El móvil envía un operation_id UUID por actualización. Si se pierde conexión, conserva la operación en almacenamiento local protegido y la reintenta con la misma clave; la pantalla la muestra pendiente hasta que el servidor la acepta. No muestra “entregado” definitivo solo por guardar localmente.

Logística valida la transición, el conductor asignado y las cantidades; deduplica operation_id con la misma protección de operaciones de negocio. Para conservar esa clave, se añade al contrato de actualización y se registra en INBOX_EVENTOS como identificador de operación de API, con un namespace UUID independiente. Las lecturas usan sensor_event_id. Las horas recibidas del dispositivo se contrastan con el servidor; datos tardíos quedan marcados para revisión, sin cambiar retroactivamente las ventanas. El refresh token se guarda en el almacén seguro del sistema operativo.

5.6. Indicadores con fórmula y fuente

Indicador	Cálculo y criterio
Merma en kg del periodo	SUM(quantity) de inventory.waste_recorded por fecha efectiva y producto. Una alerta de frío sin pérdida confirmada aporta 0 kg.
Porcentaje de merma por cohorte de cosecha	100 × kg de merma acumulada de los lotes cosechados en el periodo / kg iniciales de esos mismos lotes. Se informa fecha de corte; usa harvest.registered e inventory.waste_recorded. No divide la merma de hoy entre ventas de hoy.
Rotación física del inventario	kg despachados en el periodo / stock físico medio del periodo. El stock medio es el promedio ponderado por tiempo de los saldos de LOTES_INVENTARIO, no solo el saldo al cierre. Fuentes: movimientos y stock_changed.
Cumplimiento de ventanas	100 × entregas completed con window_start <= delivered_at <= window_end / entregas cuya window_end vence en el periodo, excluyendo únicamente canceladas antes de window_start. Parciales, fallidas, tardías y pendientes vencidas no cumplen.
Duración de entrega	Minutos entre dispatched_at y delivered_at, cuando ambas fechas existen. Se informa media y p95; no sustituye el indicador de puntualidad.
Satisfacción	AVG(score) y número de valoraciones recibidas desde rating.created. No se interpreta ausencia de valoración como una nota de cero.

Ejemplos: una cohorte de 100 kg con 8 kg de merma acumulada tiene 8% al corte. Despachar 120 kg con stock medio de 40 kg representa 3 rotaciones. Si vencían 10 entregas y 8 fueron completas dentro de su ventana, el cumplimiento es 80%; una entrega tardía no se mueve al día siguiente para mejorar ese resultado.

Los indicadores declaran periodo, unidad, zona horaria, fecha de actualización y cobertura. Si el denominador es cero o falta el saldo inicial, se muestra “sin dato”, no cero. Rotación no se calcula mezclando kg con unidades. Analítica conserva los hechos en EVENTOS_PROCESADOS y corrige agregados al recibir datos tardíos; no consulta directamente las bases operativas. Las cancelaciones posteriores al comienzo de la ventana siguen en el denominador para evitar ocultar incumplimientos.

6. Diseño del API Gateway y acceso

6.1. Enrutamiento y responsabilidades

El Gateway será la entrada HTTPS del frontend y la aplicación móvil. Conserva las rutas de Productores, Catálogo, Pedidos, Inventario, Logística, Pagos, Notificaciones y Analítica. Añade /api/v1/auth/* y /api/v1/users/* hacia Identidad y Acceso. No registra usuarios, no verifica contraseñas y no firma tokens: remite esas solicitudes a Identidad.

6.2. Endpoints de Identidad

Las rutas de esta tabla usan el prefijo /api/v1. Los campos indicados son la entrada mínima; ningún endpoint devuelve hashes de contraseñas o tokens almacenados.

Método y ruta	Acceso	Entrada y resultado
POST /auth/register	Público, limitado	email y password. Crea pending_email y rol buyer; devuelve 202 genérico y envía verificación. Rechaza roles privilegiados enviados por cliente.
POST /auth/verify-email	Token de un uso	token. Activa la cuenta verificada; 204.
POST /auth/login	Público, limitado	email, password y device_label opcional. Identidad verifica hash y estado, crea sesión y devuelve access_token, token_type, expires_in y refresh token seguro. Error genérico 401.
POST /auth/refresh	Refresh token	Rota el token de un uso y devuelve nuevo par; 401 si expirado, consumido o sesión revocada.
POST /auth/logout	Sesión propia	JWT o refresh token vigente identifica sesión. Revoca familia, borra cookie; 204.
POST /auth/logout-all	Cuenta autenticada	Revoca todas las sesiones propias, incluida la actual; 204.
GET /auth/sessions	Cuenta autenticada	Lista id, device_label, created_at y expires_at de sesiones propias; sin tokens.
DELETE /auth/sessions/{id}	Propietario	Revoca una sesión propia; 204. No permite revocar sesiones ajenas.
POST /auth/password/forgot	Público, limitado	email. Respuesta 202 uniforme; si existe cuenta, envía recuperación.
POST /auth/password/reset	Token de un uso	token y new_password. Cambia hash y revoca todas las sesiones; 204.
PUT /auth/password	Cuenta autenticada	current_password y new_password. Revalida credencial, cambia hash y revoca sesiones; nuevo login requerido.
GET /users/me	Cuenta autenticada	Devuelve id, email, status y roles de la propia cuenta.
POST /users	admin	email y roles permitidos. Crea pending_invite; envía invitación para definir contraseña, sin contraseña compartida.
POST /auth/accept-invite	Token de un uso	token y password. Verifica correo, fija credencial y activa la cuenta; 204.
PUT /users/{id}/roles	admin	roles completos. Valida vínculos operativos, aumenta roles_version y revoca sesiones de la cuenta.
PATCH /users/{id}/status	admin	active o disabled. Desactivar revoca todas las sesiones. Reactivar exige correo verificado y contraseña definida.

6.3. Endpoints internos y claves públicas

Ruta de Identidad	Uso
GET /.well-known/jwks.json	Claves públicas de firma y kid; publicación mediante ruta exacta del Gateway. Nunca entrega clave privada.
POST /internal/auth/introspect	Solo Gateway y servicios autenticados por mTLS. Recibe access_token; devuelve active, sub, sid, roles y roles_version si es válido.
GET /internal/users/{id}	Solo servicios autorizados por mTLS. Devuelve id, status y roles para validar REF; sin contraseña, hash ni sesiones.

Las rutas /internal/* no se publican ni se enrutan desde Internet. Una petición con sesión revocada recibe 401; una identidad válida sin permiso recibe 403. Los errores internos y credenciales no se incluyen en logs.

6.4. Validación y autorización

En cada ruta protegida, Gateway valida firma RS256 con algoritmo fijo, kid conocido, iss, aud, exp y nbf. Después consulta introspect para comprobar cuenta activa, sesión sid vigente y roles_version actual. Esta propuesta no almacena en caché respuestas positivas de introspección: una revocación confirmada bloquea las siguientes solicitudes. Si Identidad no está disponible, se responde 503 y no se permite el acceso protegido. El catálogo público puede continuar.

Los servicios de negocio solo aceptan conexiones del Gateway o llamadas internas autenticadas mediante mTLS. Cada servicio valida JWT y aplica autorización sobre sus recursos; si una llamada protegida no pasa por Gateway, consulta también introspect. Se eliminan encabezados de identidad aportados por el cliente. Un buyer solo accede a pedidos con buyer_id = sub; un producer se resuelve por PRODUCTORES.user_id = sub; un driver por TRANSPORTISTAS.user_id = sub y la ruta asignada.

6.5. Límites, trazas y despliegue

Gateway limita registro, login y recuperación por IP; Identidad limita también por identificador normalizado de cuenta. Se usan respuestas genéricas para no revelar si el correo existe. Se propaga request_id/correlation_id sin registrar contraseñas, Authorization, cookies ni tokens de recuperación. Identidad se despliega con su propio contenedor, migraciones, /health y base PostgreSQL en Render dentro del mismo pipeline. La clave privada solo está disponible para Identidad; Gateway y servicios reciben JWKS público. En la sección 8.6, JWT_SECRET se sustituye por JWT_PRIVATE_KEY (solo Identidad), JWT_KEY_ID, JWT_ISSUER y JWT_AUDIENCE; los verificadores configuran JWKS_URI y el algoritmo permitido.

7. Estrategia DevOps

7.1. Ramas, responsables y ambientes

Se mantienen main (versión liberable), develop (integración), feature/* y hotfix/*. Cada cambio entra por Pull Request revisado por otro integrante. main y develop exigen los estados obligatorios lint, typecheck, unit, integration y build; no se permite merge con controles omitidos o fallidos. Los PR no reciben secretos de producción. El código de un PR externo se ejecuta como código no confiable, sin permisos de publicación.

Evento	Acción obligatoria	Destino
Pull Request a develop o main	Instalar dependencias y ejecutar lint, tipos, unitarias, integración y compilación.	Sin despliegue productivo.
Push a develop	Repetir CI sobre el SHA real; construir imagen, publicar, desplegar y ejecutar smoke/carga.	Staging aislado.

Evento	Acción obligatoria	Destino
Push a main	Repetir CI sobre el SHA de merge; crear candidato, probar ese digest en staging y promoverlo si aprueba.	Producción después de staging.
workflow_dispatch de recuperación	Seleccionar servicio y digest de una versión ya validada; verificar compatibilidad y registrar motivo.	Rollback controlado.

Desarrollo usa Docker Compose. Staging y producción tienen bases PostgreSQL, RabbitMQ, cachés, claves y cuentas de prueba separados. Staging usa una pasarela simulada y datos sintéticos; las pruebas de carga no generan cobros ni mensajes a usuarios reales. No se ejecutan pruebas masivas en producción.

7.2. Un solo proceso de liberación por ambiente

GitHub Actions serializa el tramo completo de despliegue y validación mediante concurrency por ambiente, con cancel-in-progress: false. Staging queda reservado hasta terminar smoke y carga: otro push no puede cambiarlo durante la prueba. Producción también se serializa. Un manifiesto de versión lista el SHA, digest y configuración de todos los servicios; la prueba no mezcla versiones sin registrar. Los componentes no modificados conservan el digest anterior.

Se propone un monorepositorio con apps/gateway, apps/identity, los ocho servicios de negocio, apps/web y tests/load. Una matriz ejecuta CI para cada servicio afectado; cambios en librerías compartidas, contratos o lockfile vuelven a comprobar todos sus consumidores. No se usa una condición por rutas que deje un control requerido permanentemente pendiente: el job de resumen siempre comunica éxito, fallo o ausencia validada de cambios.

7.3. Docker y evidencia

Cada backend tiene Dockerfile multietapa: npm ci, compilación y una imagen final con dependencias necesarias, usuario sin privilegios y CMD de inicio. Versiones de Node, dependencias y acciones se fijan en el repositorio; acciones externas se fijan por SHA completo. Se construye linux/amd64 para Render. Las migraciones y la CLI de Prisma que requiere el pre-deploy deben estar presentes en la imagen final. [D1, D6]

Los reportes de pruebas, cobertura, carga y manifiesto de despliegue se conservan como artefactos de GitHub Actions durante 30 días. El manifiesto incluye servicio, commit, digest, deploy_id, ambiente, hora, resultado y versión anterior. Los logs eliminan credenciales y datos personales. Esta sección define el pipeline que deberá implementarse; no afirma que se hayan ejecutado estas pruebas sobre código no aportado.

8. Pipeline CI/CD con GitHub Actions

8.1. Controles obligatorios de CI

Los siguientes scripts se definen en package.json y se ejecutan en el directorio de cada aplicación afectada. Ninguno utiliza --if-present ni continue-on-error. npm ci debe terminar antes de los demás controles; cualquier código de salida distinto de cero bloquea publicación y despliegue.

Control / comando	Criterio de aceptación y evidencia
npm ci	Instalación reproducible desde package-lock.json; falla si el archivo no coincide con package.json.
npm run lint	Script: eslint . --max-warnings=0. Incluye TypeScript, pruebas y scripts de CI; cero errores y cero advertencias.
npm run typecheck	Script: tsc --noEmit. Cero errores de tipos, separado del lint.
npm run test:unit -- --ci --coverage	Jest; todas las pruebas pasan. Meta propuesta: >= 80% de líneas, funciones, sentencias y ramas por servicio. Reporte JUnit y cobertura; no se excluye lógica crítica para alcanzar el porcentaje.

Control / comando	Criterio de aceptación y evidencia
npm run test:integration -- --ci --runInBand	Jest + Supertest contra PostgreSQL y RabbitMQ efímeros reales, y Redis cuando aplique. Todas pasan; sin reutilizar la base de desarrollo.
npm run build	Compila cada backend y el frontend afectado. No sustituye lint ni pruebas. Se verifica que Docker arranque con su CMD.

8.2. Casos mínimos ligados al dominio

Nivel	Casos que se deben demostrar
Unitarias	FEFO; mínimo de compra; precio según stock; bruto/comisión/neto y redondeo; transiciones de pedido; permisos por propietario; expiración y reutilización de tokens.
Integración de datos	Migrar una base vacía y una copia de esquema de la versión anterior; probar PK/FK, UNIQUE y CHECK; crear, reservar y liberar lotes mediante API.
Concurrencia	Dos solicitudes disputan la última unidad: no hay stock negativo ni doble reserva. Los totales del pedido y la liquidación coinciden con sus detalles.
Eventos y Saga	Publicar dos veces el mismo evento no duplica cobros, mermas ni valoraciones. Fallo de autorización cancela y libera reserva; probar reintento y confirmación del mensaje después del commit.
Identidad y contratos	Cuenta desactivada no opera; refresh consumido revoca su familia. Validar esquemas versionados de eventos y respuestas entre servicios.

En integración, la base y el broker no se sustituyen por mocks; la pasarela de pagos y correo externos sí se simulan. Antes de arrancar las pruebas se esperan sus controles de disponibilidad, se aplican migraciones y se cargan datos sintéticos. Al finalizar se destruye el entorno aunque haya fallos.

8.3. Pruebas de carga en staging

Se incorpora k6 como control de promoción. Se proponen los siguientes valores iniciales; son metas de aceptación por validar con el equipo, no resultados medidos ni garantías de capacidad. k6 devuelve fallo si incumple un threshold, lo que impide promover la imagen. [D7]

Perfil	Carga y duración	Objetivo
Calentamiento y carga habitual	0 a 20 usuarios virtuales en 1 minuto; 20 usuarios durante 3 minutos.	Estabilizar conexiones, caché y consultas.
Pico de pedidos	20 a 100 usuarios en 2 minutos; mantener 100 durante 5 minutos; bajar a 0 en 1 minuto.	Simular el pico de compras señalado en el proyecto.
Distribución	70% consultas de catálogo; 20% carrito; 10% creación/consulta de pedidos.	Usar usuarios, productos y lotes distintos; cuentas y tokens preparados antes del pico.

Métrica	Umbral obligatorio propuesto
Errores HTTP inesperados	http_req_failed: rate < 0.01; cuentas autorizadas y stock suficiente en el perfil de capacidad.
Latencia de catálogo	http_req_duration{scenario:catalog}: p(95) < 500 ms.
Aceptación de pedido	http_req_duration{scenario:orders}: p(95) < 1500 ms; mide POST /orders, no la entrega física.
Confirmación asíncrona	Métrica propia order_confirmation_ms: p(95) < 5000 ms desde aceptación hasta confirmación con pasarela simulada.

Métrica	Umbral obligatorio propuesto
Invariantes	0 reservas/cobros duplicados y 0 saldos negativos en comprobación posterior. Pedidos iniciados = confirmados + cancelados; ninguno queda pendiente después de 30 segundos de drenaje.

Los scripts usan escenarios etiquetados catalog, cart y orders para que los umbrales se apliquen al tráfico correcto. El perfil de capacidad no provoca rechazos esperados por falta de stock; la disputa del último lote se prueba separadamente y verifica el rechazo correcto. Se registra CPU, memoria, conexiones de PostgreSQL y profundidad de colas para explicar un incumplimiento. Si hay muy pocas muestras, un escenario no se ejecutó o la prueba se interrumpe, el resultado no se considera aprobado.

```
k6 run --summary-export=artifacts/k6-summary.json \
  tests/load/marketplace.js
```

Se guardan versión de k6, configuración del ambiente, fixture, SHA y digests de la prueba. La promoción usa exactamente esos digests. Si el ambiente tiene menos recursos que producción, el informe lo declara; no extrapola capacidad solo a partir del número de usuarios virtuales. Un ajuste de umbrales requiere cambio revisado en el repositorio.

8.4. Imágenes y activación de Render

Cada imagen se publica como ghcr.io/<organizacion>/farmtotable-<servicio>:sha-<SHA_COMPLETO>. Las etiquetas semánticas vX.Y.Z son opcionales y solo se agregan a una versión aceptada. La política impide sobrescribir la etiqueta de un SHA; el despliegue se fija por digest ghcr.io/<organizacion>/farmtotable-<servicio>@sha256:<DIGEST>, que identifica los bytes probados. No se usa latest para producción o recuperación.

GitHub Actions publica con GITHUB_TOKEN y permiso packages: write en el job de publicación; CI ordinaria solo necesita contents: read. Render recibe una credencial independiente de lectura de GHCR con read:packages. Se conserva el digest activo y al menos las cinco versiones estables anteriores durante 90 días, y no se elimina una imagen todavía usada. [D1, D6]

Paso que inicia el despliegue

Se elige la API de Render como mecanismo único: el job deploy_render depende de CI, build/push y de los controles de staging cuando el destino es producción. Ejecuta POST /v1/services/{serviceId}/deploys con imageUrl fijada al digest. Host, repositorio y nombre de imagen deben coincidir con la configuración del servicio. El push a GHCR solo almacena la imagen; no actualiza un servicio basado en imágenes. [D1, D2]

```
jq -n --arg image "$IMAGE_REF" \
  '{imageUrl: $image}' > deploy-request.json

curl --silent --show-error --fail-with-body \
  -X POST \
  -H "Authorization: Bearer $RENDER_API_KEY" \
  -H "Content-Type: application/json" \
  --data-binary @deploy-request.json \
  "https://api.render.com/v1/services/$RENDER_SERVICE_ID/deploys" \
  > deploy-response.json
```

El fragmento pertenece al job de despliegue; IMAGE_REF proviene de la salida validada de build/push, no de una etiqueta mutable. RENDER_SERVICE_ID se selecciona de una matriz fija por servicio y ambiente. RENDER_API_KEY es secreto de GitHub Environment y no se imprime; tampoco se usa set -x. Este documento no contiene valores reales ni activa ningún despliegue.

Esperar y comprobar el resultado

La respuesta aceptada no significa que la versión esté activa. Se conserva el deploy_id y se consulta GET /v1/services/{serviceId}/deploys/{deployId} cada 10 segundos, hasta estado live, fallo/cancelación terminal o límite local de 25 minutos. Un 202 en cola exige resolver y seguir el despliegue exacto; sin un ID verificable no se declara éxito. Después de live, una prueba autenticada consulta /internal/version por un acceso de CI restringido y compara el SHA con el manifiesto. [D2, D3]

Si se pierde la respuesta del POST, primero se reconcilia la lista de despliegues con servicio, imagen y hora; no se repite ciegamente una operación que pudo haber empezado. Un timeout del job no cancela por sí solo Render: se detiene la promoción y se revisa el estado real antes de otro intento. El acceso de CI usa un proxy operativo autenticado del Gateway, limitado a su identidad técnica; /internal/version no se expone al público.

8.5. Diagrama del pipeline y secretos

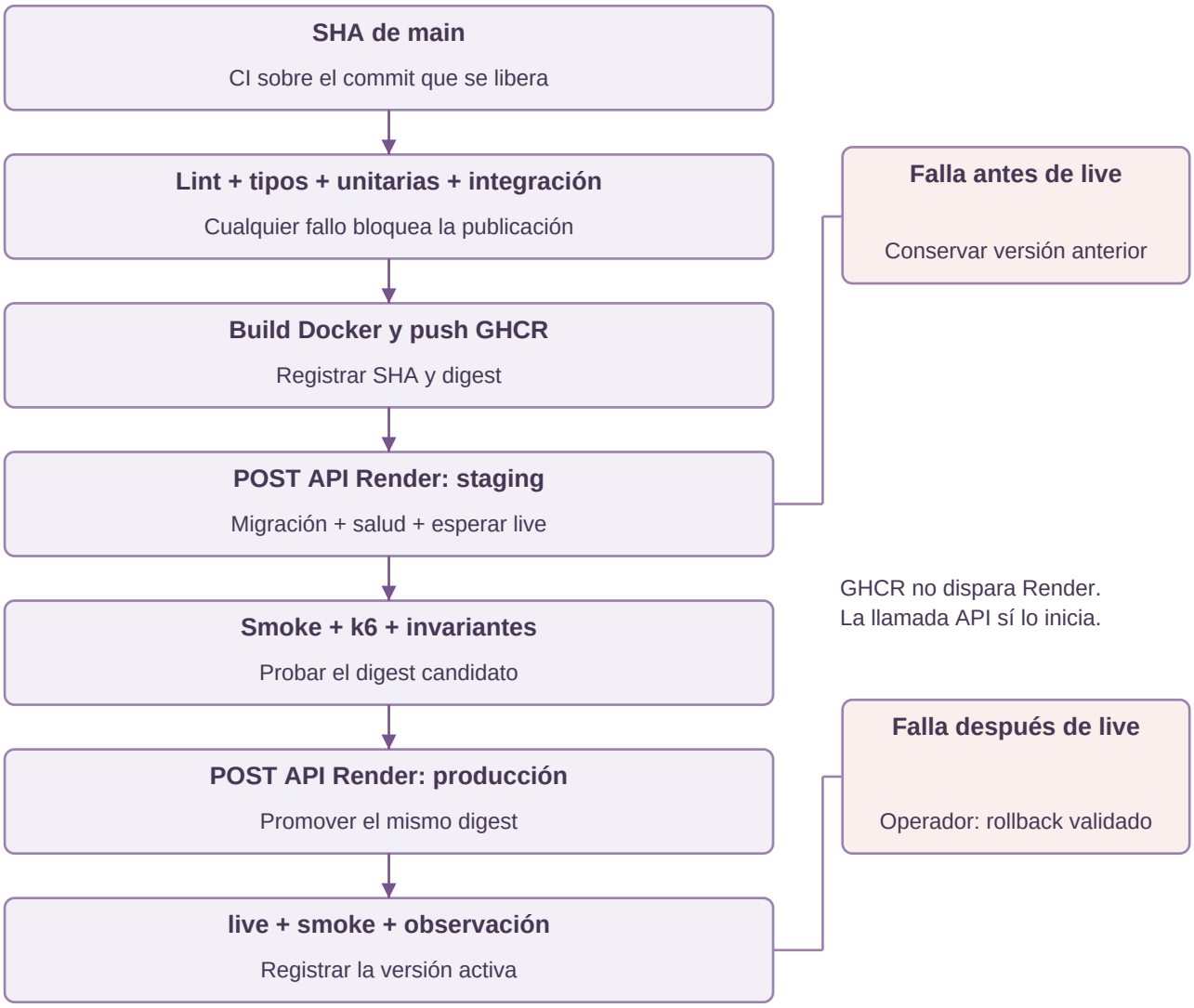


Figura 11. Camino de liberación desde main y tratamiento de fallos. Un PR solo ejecuta CI; develop termina en staging. En producción, el responsable de liberación puede ejecutar la recuperación del último digest estable cuando la evaluación de la incidencia lo justifique.

8.6. Variables y secretos por ambiente

Dato	Ubicación y acceso
RENDER_API_KEY; VERCEL_TOKEN	Secretos de GitHub Environment; solo jobs de despliegue.
RENDER_SERVICE_ID; IMAGE_REF; VERCEL_ORG_ID; VERCEL_PROJECT_ID	Variables o salidas validadas; mapeo separado para staging y producción.
DATABASE_URL; RABBITMQ_URL; REDIS_URL	Secretos de Render propios de cada servicio; no se incorporan a la imagen.
JWT_PRIVATE_KEY; PAYMENT_API_KEY; credenciales de correo	Secretos de Identidad/Pagos según propiedad; nunca accesibles a CI de PR.
NEXT_PUBLIC_API_URL	Configuración pública del frontend por ambiente; no puede contener secretos.

9. Estrategia de despliegue

9.1. Topología y cambio de versión

Gateway es un web service público de Render. Identidad y los ocho servicios de negocio son private services en la misma región, con PostgreSQL independiente por servicio y sin discos persistentes adjuntos a los contenedores de aplicación. Se presupone un plan que permita pre-deploy. El frontend permanece en Vercel. No se requiere AWS, Azure ni Google Cloud.

Se precisa la etiqueta anterior “Rolling Deployment”: se usa el reemplazo sin interrupción de Render, no un reparto canary de porcentajes. La versión anterior atiende mientras inicia la nueva. Con una instancia, el tráfico cambia al estar lista; con varias, Render reemplaza instancias de forma progresiva. La aplicación atiende SIGTERM, deja de tomar mensajes, termina o devuelve trabajos pendientes y cierra conexiones antes de salir. Se configuran 30 segundos para cierre y operaciones compatibles con ese margen. [D4]

9.2. Salud antes y después de activar

Render permite health checks HTTP en web services y TCP en private services. Para Gateway se configura /health/ready; el proyecto devuelve 200 o 503, sin autenticación y sin datos sensibles. Render acepta 2xx/3xx en menos de 5 segundos. En un despliegue nuevo, si las instancias no superan los controles dentro de 15 minutos, cancela el intento y conserva las anteriores; en el primer despliegue no existe una versión previa que mantener. [D5]

Para cada private service, el proceso completa inicialización y verificación de su esquema antes de abrir el puerto que Render sondea. Se mantiene /health/ready para monitoreo HTTP interno, pero no se afirma que Render lo consulte. El smoke de CI y el monitoreo interno comprueban disponibilidad funcional y versión. Un puerto abierto por sí solo no detecta errores de negocio.

La readiness comprueba el proceso, una consulta corta a su propia base y compatibilidad de esquema. RabbitMQ caído puede marcar degradación sin detener escrituras que se guardan en outbox; Redis no bloquea si hay lectura de PostgreSQL. No se encadenan todos los servicios en /health: una caída de Analítica no debe provocar reinicios de Pedidos. Las operaciones cuya dependencia sea imprescindible devuelven 503 según el contrato, como autenticación sin Identidad.

Situación	Interpretación y acción
Candidata falla antes de live	Despliegue rechazado. La versión anterior sigue atendiendo; no se registra como rollback posterior.
Instancia activa falla salud	Render retira tráfico tras fallos consecutivos de 15 segundos y reinicia tras 60 segundos. Reinicio no significa volver a otra imagen. [D5]
Versión live falla smoke o métricas	Incidencia posterior a activación. El operador evalúa y dispara un rollback explícito a una versión validada.

9.3. Migraciones previas al arranque

Se configura pre-deploy command: npx prisma migrate deploy. Render lo ejecuta con la imagen candidata antes de iniciar su despliegue; si falla, no continúa. La migración actúa únicamente sobre la base del servicio. No se ejecuta dentro del arranque de cada réplica. [D4]

Se aplica expandir y después retirar: primero columnas/tablas compatibles, después código que las usa, y en una liberación posterior la eliminación de estructuras antiguas. Mientras conviven dos versiones, ambas deben leer y escribir correctamente. Una migración ya aplicada no se deshace porque la candidata falle salud. Antes de cambios de alto impacto se valida restauración en staging y se conserva respaldo; restaurar datos no es el rollback habitual de código.

9.4. Validación posterior y rollback

Después de live, CI ejecuta durante dos minutos pruebas sintéticas con cuentas dedicadas: login, consulta de catálogo y reserva/cancelación controlada. Se comprueba el SHA esperado. Se propone además observar diez minutos de errores 5xx, latencias y colas: error > 1% o p95 > 2 veces la línea base durante cinco minutos consecutivos generan alerta; un incumplimiento de integridad genera alerta inmediata. Son umbrales operativos propuestos, no garantías de Render.

El responsable de liberación congela nuevas promociones y distingue fallo de código de caída de una dependencia. Si el problema está en la nueva versión y la anterior sigue siendo compatible con esquema y eventos, ejecuta Rollback hacia el deploy_id estable guardado en el manifiesto, desde Render o el workflow de recuperación. Esto inicia otro despliegue. Si la causa es PostgreSQL o RabbitMQ, cambiar de imagen puede no resolverla; se atiende la dependencia.

Para imágenes externas, Render vuelve a obtener la referencia de destino desde el registro. Por eso se conserva el digest; una etiqueta sobrescrita podría recuperar otros bytes y una imagen eliminada impediría volver. Se verifican configuración, secretos y compatibilidad de esquema antes de iniciar. El rollback no restaura PostgreSQL ni revierte operaciones de negocio. [D8]

Después de recuperar, se espera live, se comprueba versión y se repite smoke. Se registra el deploy_id recuperado y se bloquea el digest defectuoso para futuras promociones. Si falla la recuperación, se mantiene la gestión de la incidencia; no se declara restaurado el servicio. No se promete un rollback automático solo porque un health check de una versión activa falle.

9.5. Despliegue del frontend en Vercel

GitHub Actions controla también el despliegue del frontend; se deshabilita el despliegue Git automático de producción para evitar que un push salte los controles de CI. Una vez aprobado el backend candidato, el job del frontend ejecuta vercel pull --yes --environment=production, vercel build --prod y vercel deploy --prebuilt --prod, con la CLI fijada y el token del ambiente. Se guarda el ID/URL devuelto y se valida la navegación y conexión al Gateway. Para preview se usa la configuración preview y se omite --prod. [D9]

Staging y producción pueden requerir compilaciones distintas del frontend porque NEXT_PUBLIC_API_URL se incorpora al build; ambas usan el mismo SHA verificado y cada artefacto queda identificado. Esta excepción no reconstruye las imágenes del backend: estas se promueven por digest. Si falla el frontend activo, se selecciona el deployment productivo anterior validado en Vercel y se comprueba su compatibilidad con /api/v1.

9.6. Compatibilidad de APIs y alcance

Se mantiene /api/v1 para cambios compatibles; un contrato incompatible requiere /api/v2 o una transición compatible documentada. Los eventos incorporan schema_version y consumidores tolerantes a campos nuevos. Se despliegan primero consumidores compatibles y después productores del contrato nuevo. No se actualizan todos los servicios como una única transacción: el manifiesto permite identificar y recuperar cada componente.

El documento define comandos, condiciones y responsabilidades; no acredita ejecuciones de CI, resultados k6 ni despliegues reales. La implementación deberá aportar workflows, scripts, Dockerfiles, IDs de servicio y evidencia de las pruebas descritas.

9.7. Referencias técnicas verificadas

Documentación oficial consultada para precisar la propuesta. Los valores de cobertura, carga, latencia, retención y observación son decisiones de diseño de FarmToTable.

[D1] Render - Deploy a Prebuilt Docker Image

[D2] Render API - Trigger deploy

[D3] Render API - Retrieve deploy

[D4] Render - Deploying on Render

[D5] Render - Health Checks

[D6] GitHub - Publishing Docker images

[D7] Grafana k6 - Thresholds

[D8] Render - Rollbacks

[D9] Vercel - GitHub Actions integration

[D10] Render - Web Services

10. Observabilidad y monitoreo

Cada servicio genera logs estructurados con fecha, nivel, servicio, request_id/correlation_id y versión desplegada. No registra contraseñas, JWT, cookies ni datos de tarjetas. La búsqueda centralizada permite seguir un pedido desde la reserva hasta la captura y su entrega, sin entrar en cada contenedor.

Prometheus reúne latencias, errores, uso de recursos, conexiones y colas; Grafana presenta paneles. Las alertas distinguen una instancia caída, una dependencia no disponible y una regresión de negocio. Se vigilan reservas vencidas, Sagas pendientes, diferencias de stock, callbacks de pago sin conciliar y eventos no procesados.

Gateway usa /health/ready como check HTTP de Render. Los servicios privados usan el check TCP de Render y readiness HTTP interna, conforme a 9.2. CI verifica versión y smoke después de live; el monitoreo vigila salud y métricas después. Un reinicio por salud no equivale a rollback de imagen. Los indicadores de operación se definen en 5.6 y no se confunden con métricas técnicas como CPU.

11. Seguridad arquitectónica

11.1. Emisión y validación de tokens

Identidad y Acceso es el emisor único. Tras verificar las credenciales y el estado de cuenta, crea SESIONES y firma el access token con RS256 usando una clave privada disponible solo en ese servicio. El JWT contiene sub = USUARIOS.id, sid = SESIONES.id, roles, roles_version, iss, aud, iat, nbf, exp y jti. No contiene contraseña ni otros secretos. Gateway y servicios verifican la firma con las claves públicas de JWKS y el algoritmo configurado; nunca eligen un algoritmo libremente a partir del token.

Se proponen access tokens de 10 minutos, sesiones con límite absoluto de 30 días y refresh tokens con vencimiento a los 7 días o al límite de sesión, lo que ocurra primero. Son parámetros del proyecto. La rotación de claves usa kid y mantiene temporalmente la clave pública anterior hasta que expiren los JWT emitidos con ella. La clave privada no se comparte con Gateway ni con los servicios de negocio. [S1]

11.2. Contraseñas, verificación y recuperación

Identidad almacena password_hash con Argon2id, salt aleatorio por contraseña y parámetros codificados en el hash. Se propone como punto de partida m = 19 MiB, t = 2 y p = 1, sujeto a medición en el entorno. Esta decisión reemplaza la mención anterior a bcrypt. Solo Identidad compara credenciales; los demás servicios no reciben hashes. El registro verifica el correo antes de habilitar acceso. [S2]

Los tokens de verificación, invitación y recuperación son secretos aleatorios de un solo uso, almacenados únicamente como SHA-256 en TOKENS_CUENTA. Se propone vigencia de 24 horas para verificación/invitación y 30 minutos para recuperación. Solicitar otro invalida el anterior del mismo propósito. Cambiar o restablecer una contraseña revoca todas las sesiones y obliga a iniciar sesión otra vez. Las respuestas de recuperación son uniformes, exista o no el correo. No se registran enlaces o tokens en logs.

11.3. Sesiones y rotación de refresh tokens

Cada login crea una sesión independiente por dispositivo. El refresh token es opaco, aleatorio de 256 bits y solo su SHA-256 se almacena en TOKENS_REFRESH. Al renovar, Identidad bloquea la cuenta, sesión y token en un orden fijo, comprueba estado, vencimiento y uso previo; consume el token anterior y crea su sucesor en una transacción. El límite absoluto de la sesión no se extiende. El cliente serializa las renovaciones para no reutilizar el mismo token por solicitudes concurrentes.

Si reaparece un refresh token ya consumido, Identidad revoca su sesión completa, conserva la cadena como evidencia y exige nuevo login; no entrega otro token. La detección sigue el principio de rotación y detección de reutilización descrito en RFC 9700; estos endpoints propios no se presentan como una implementación completa de OAuth u OpenID Connect. [S3]

Logout revoca la sesión actual; logout-all revoca todas las del usuario. Desactivar una cuenta o cambiar sus roles también revoca sus sesiones. Identidad comprueba esos cambios dentro de la misma transacción de renovación para evitar emitir tokens después de una baja. Los refresh tokens consumidos se conservan hasta el vencimiento absoluto de la familia para detectar reutilización. Gateway consulta introspección en cada petición protegida: los JWT aún no vencidos de sesiones revocadas también son rechazados. Una operación ya autorizada antes de la revocación puede terminar; la baja no deshace transacciones confirmadas.

En web, el refresh token se entrega en cookie Secure y HttpOnly, restringida a /api/v1/auth y sin dominio amplio. Si frontend y API usan sitios diferentes, se requiere SameSite=None; si comparten sitio, se usa Lax o Strict. En ambos casos, las operaciones con cookie verifican Origin contra una lista exacta y usan defensa CSRF. El access token se mantiene en memoria. En móvil, el refresh token se conserva en el almacenamiento seguro del sistema y se envía por HTTPS; no se guarda en archivos planos.

11.4. Roles y propiedad del recurso

Rol	Permisos y límites
buyer	Crear pedidos y consultar únicamente compras propias; valorar pedidos entregados.
producer	Administrar el perfil, fincas, productos y lotes vinculados a PRODUCTORES.user_id = sub.
driver	Consultar y actualizar entregas de rutas asignadas al perfil TRANSPORTISTAS.user_id = sub.
logistics_admin	Administrar transportistas operativos, vehículos y rutas. No conceder roles de acceso.
admin	Gestionar cuentas, estados y roles mediante endpoints auditados. No consultar contraseñas o hashes.

Un rol permite realizar una clase de operación; no elimina la comprobación de propiedad del recurso. El registro público no acepta admin, driver, producer ni logistics_admin. Los cambios administrativos se registran con actor, usuario afectado, acción, fecha y correlation_id, sin secretos, en los logs estructurados y su almacenamiento centralizado. Se impide desactivar o retirar el rol al último admin activo.

11.5. Controles de aplicación y transporte

SQL Injection: Prisma usa consultas parametrizadas y se validan entradas. XSS: el frontend escapa contenido y evita HTML no confiable. CSRF: las operaciones basadas en cookies validan origen y token anti-CSRF. CORS: solo orígenes exactos autorizados, sin comodín con credenciales. HTTPS protege tráfico externo; mTLS autentica Gateway y servicios en rutas internas. Los microservicios no exponen públicamente sus bases ni endpoints internos.

Las claves de PostgreSQL, RabbitMQ, firma y correo se conservan en secretos de despliegue, nunca en código. Cada servicio usa credenciales con acceso exclusivo a su base. Gateway aplica rate limiting y timeouts; Identidad añade limitación por cuenta y registro de intentos fallidos sin credenciales. Se mantienen actualizaciones de dependencias, copias de seguridad y alertas por fallos y accesos sospechosos. Si introspección no está disponible, el acceso protegido falla de forma cerrada con 503; esta dependencia debe incluirse en el monitoreo y escalado de Identidad.

Referencias de seguridad

[S1] OWASP - REST Security Cheat Sheet

[S2] OWASP - Password Storage Cheat Sheet

[S3] RFC 9700, sección 4.9 - Refresh Token Protection

Las fuentes respaldan los principios de validación, hash y rotación. La separación de componentes, endpoints, roles, plazos y esquema relacional son decisiones propuestas para FarmToTable.

Conclusión

FarmToTable queda organizado en nueve microservicios con bases relacionales independientes. Identidad y Acceso administra cuentas, credenciales y sesiones; Gateway valida y enruta; los servicios de negocio autorizan sobre sus propios recursos. Se conserva la comunicación REST y RabbitMQ, el despliegue con Docker y el pipeline de GitHub Actions, Render y Vercel.